JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Character

java.lang.Object
    java.lang.Character

All Implemented Interfaces:

`Serializable`, `Comparable<Character>`, `Constable`

```
public final class Character
extends Object
implements Serializable, Comparable<Character>, Constable
```

The `Character` class is the wrapper class for values of the primitive type `char`. An object of type `Character` contains a single field whose type is `char`.

In addition, this class provides a large number of static methods for determining a character's category (lowercase letter, digit, etc.) and for converting characters from uppercase to lowercase and vice versa.

Unicode Conformance

The fields and methods of class `Character` are defined in terms of character information from the Unicode Standard, specifically the *UnicodeData* file that is part of the Unicode Character Database. This file specifies properties including name and category for every assigned Unicode code point or character range. The file is available from the Unicode Consortium at http://www.unicode.org.

Character information is based on the Unicode Standard, version 17.0.

The Java platform has supported different versions of the Unicode Standard over time. The following tables list the version of Unicode used in each Java release. Unless otherwise specified, all update releases in a given Java release family use the same Unicode version.

Java release	Unicode version
Java SE 26	Unicode 17.0
Java SE 25	Unicode 16.0
Java SE 21	Unicode 15.0
Java SE 17	Unicode 13.0
Java SE 11	Unicode 10.0
Java SE 8	Unicode 6.2

Show other Java releases

Java releases prior to Java SE 8 are listed only if they upgraded the Unicode version

Java release	Unicode version
Java SE 24	Unicode 16.0
Java SE 23	Unicode 15.1
Java SE 22	Unicode 15.1
Java SE 20	Unicode 15.0
Java SE 19	Unicode 14.0
Java SE 18	Unicode 13.0
Java SE 16	Unicode 13.0
Java SE 15	Unicode 13.0
Java SE 14	Unicode 12.1
Java SE 13	Unicode 12.1
Java SE 12	Unicode 11.0
Java SE 10	Unicode 8.0
Java SE 9	Unicode 8.0
Java SE 7	Unicode 6.0
Java SE 5.0	Unicode 4.0
Java SE 1.4	Unicode 3.0
JDK 1.1	Unicode 2.0
JDK 1.0.2	Unicode 1.1.5

Variations from these base Unicode versions, such as recognized appendixes, are documented elsewhere.

Unicode Character Representations

The `char` data type (and therefore the value that a `Character` object encapsulates) are based on the original Unicode specification, which defined characters as fixed-width 16-bit entities. The Unicode Standard has since been changed to allow for characters whose representation requires more than 16 bits. The range of legal *code point*s is now U+0000 to U+10FFFF, known as *Unicode scalar value*.

The set of characters from U+0000 to U+FFFF is sometimes referred to as the *Basic Multilingual Plane (BMP)*. Characters whose code points are greater than U+FFFF are called *supplementary character*s. The Java platform uses the UTF-16 representation in `char` arrays and in the `String` and `StringBuffer` classes. In this representation, supplementary characters are represented as a pair of `char` values, the first from the *high-surrogates* range, (\uD800-\uDBFF), the second from the *low-surrogates* range (\uDC00-\uDFFF).

A `char` value, therefore, represents Basic Multilingual Plane (BMP) code points, including the surrogate code points, or code units of the UTF-16

encoding. An `int` value represents all Unicode code points, including supplementary code points. The lower (least significant) 21 bits of `int` are used to represent Unicode code points and the upper (most significant) 11 bits must be zero. Unless otherwise specified, the behavior with respect to supplementary characters and surrogate `char` values is as follows:

- The methods that only accept a `char` value cannot support supplementary characters. They treat `char` values from the surrogate ranges as undefined characters. For example, `Character.isLetter('\uD840')` returns `false`, even though this specific value if followed by any low-surrogate value in a string would represent a letter.
- The methods that accept an `int` value support all Unicode characters, including supplementary characters. For example, `Character.isLetter(0x2F81A)` returns `true` because the code point value represents a letter (a CJK ideograph).

In the Java SE API documentation, *Unicode code point* is used for character values in the range between U+0000 and U+10FFFF, and *Unicode code unit* is used for 16-bit `char` values that are code units of the *UTF-16* encoding. For more information on Unicode terminology, refer to the Unicode Glossary.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail.

Since:

1.0

External Specifications

Unicode Character Database

See Also:

Serialized Form

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
`static class`	`Character.Subset`	Instances of this class represent particular subsets of the Unicode character set.
`static final class`	`Character.UnicodeBlock`	A family of character subsets representing the

		character blocks in the Unicode specification.
static enum	Character.UnicodeScript	A family of character subsets representing the character scripts defined in the *Unicode Standard Annex #24: Script Names*.

Field Summary

Fields

Modifier and Type	Field	Description
static final int	BYTES	The number of bytes used to represent a `char` value in unsigned binary form.
static final byte	COMBINING_SPACING_MARK	General category "Mc" in the Unicode specification.
static final byte	CONNECTOR_PUNCTUATION	General category "Pc" in the Unicode specification.
static final byte	CONTROL	General category "Cc" in the Unicode specification.
static final byte	CURRENCY_SYMBOL	General category "Sc" in the Unicode specification.
static final byte	DASH_PUNCTUATION	General category "Pd" in the Unicode specification.
static final byte	DECIMAL_DIGIT_NUMBER	General category "Nd" in the Unicode specification.

static final byte	DIRECTIONALITY_ARABIC_NU	Weak bidirectional character type "AN" in the Unicode specification.
static final byte	DIRECTIONALITY_BOUNDARY_	Weak bidirectional character type "BN" in the Unicode specification.
static final byte	DIRECTIONALITY_COMMON_NU	Weak bidirectional character type "CS" in the Unicode specification.
static final byte	DIRECTIONALITY_EUROPEAN_	Weak bidirectional character type "EN" in the Unicode specification.
static final byte	DIRECTIONALITY_EUROPEAN_	Weak bidirectional character type "ES" in the Unicode specification.
static final byte	DIRECTIONALITY_EUROPEAN_	Weak bidirectional character type "ET" in the Unicode specification.
static final byte	DIRECTIONALITY_FIRST_STR	Weak bidirectional character type "FSI" in the Unicode specification.
static final byte	DIRECTIONALITY_LEFT_TO_R	Strong bidirectional character type "L" in the Unicode specification.
static final byte	DIRECTIONALITY_LEFT_TO_R	Strong bidirectional character type "LRE" in the Unicode specification.
static final byte	DIRECTIONALITY_LEFT_TO_R	Weak bidirectional character type "LRI" in

		the Unicode specification.
static final byte	DIRECTIONALITY_LEFT_TO_R	Strong bidirectional character type "LRO" in the Unicode specification.
static final byte	DIRECTIONALITY_NONSPACIN	Weak bidirectional character type "NSM" in the Unicode specification.
static final byte	DIRECTIONALITY_OTHER_NEU	Neutral bidirectional character type "ON" in the Unicode specification.
static final byte	DIRECTIONALITY_PARAGRAPH	Neutral bidirectional character type "B" in the Unicode specification.
static final byte	DIRECTIONALITY_POP_DIREC	Weak bidirectional character type "PDF" in the Unicode specification.
static final byte	DIRECTIONALITY_POP_DIREC	Weak bidirectional character type "PDI" in the Unicode specification.
static final byte	DIRECTIONALITY_RIGHT_TO_	Strong bidirectional character type "R" in the Unicode specification.
static final byte	DIRECTIONALITY_RIGHT_TO_	Strong bidirectional character type "AL" in the Unicode specification.
static final byte	DIRECTIONALITY_RIGHT_TO_	Strong bidirectional character type "RLE" in

		the Unicode specification.
static final byte	DIRECTIONALITY_RIGHT_TO_	Weak bidirectional character type "RLI" in the Unicode specification.
static final byte	DIRECTIONALITY_RIGHT_TO_	Strong bidirectional character type "RLO" in the Unicode specification.
static final byte	DIRECTIONALITY_SEGMENT_S	Neutral bidirectional character type "S" in the Unicode specification.
static final byte	DIRECTIONALITY_UNDEFINED	Undefined bidirectional character type.
static final byte	DIRECTIONALITY_WHITESPAC	Neutral bidirectional character type "WS" in the Unicode specification.
static final byte	ENCLOSING_MARK	General category "Me" in the Unicode specification.
static final byte	END_PUNCTUATION	General category "Pe" in the Unicode specification.
static final byte	FINAL_QUOTE_PUNCTUATION	General category "Pf" in the Unicode specification.
static final byte	FORMAT	General category "Cf" in the Unicode specification.
static final byte	INITIAL_QUOTE_PUNCTUATIO	General category "Pi" in the Unicode specification.

`static final byte`	**`LETTER_NUMBER`**	General category "Nl" in the Unicode specification.
`static final byte`	**`LINE_SEPARATOR`**	General category "Zl" in the Unicode specification.
`static final byte`	**`LOWERCASE_LETTER`**	General category "Ll" in the Unicode specification.
`static final byte`	**`MATH_SYMBOL`**	General category "Sm" in the Unicode specification.
`static final int`	**`MAX_CODE_POINT`**	The maximum value of a Unicode code point, constant `U+10FFFF`.
`static final char`	**`MAX_HIGH_SURROGATE`**	The maximum value of a Unicode high-surrogate code unit in the UTF-16 encoding, constant `'\uDBFF'`.
`static final char`	**`MAX_LOW_SURROGATE`**	The maximum value of a Unicode low-surrogate code unit in the UTF-16 encoding, constant `'\uDFFF'`.
`static final int`	**`MAX_RADIX`**	The maximum radix available for conversion to and from strings.
`static final char`	**`MAX_SURROGATE`**	The maximum value of a Unicode surrogate code unit in the UTF-16 encoding, constant `'\uDFFF'`.
`static final char`	**`MAX_VALUE`**	The constant value of this field is the largest value of type `char`, `'\uFFFF'`.

`static final int`	**`MIN_CODE_POINT`**	The minimum value of a Unicode code point, constant `U+0000`.
`static final char`	**`MIN_HIGH_SURROGATE`**	The minimum value of a Unicode high-surrogate code unit in the UTF-16 encoding, constant `'\uD800'`.
`static final char`	**`MIN_LOW_SURROGATE`**	The minimum value of a Unicode low-surrogate code unit in the UTF-16 encoding, constant `'\uDC00'`.
`static final int`	**`MIN_RADIX`**	The minimum radix available for conversion to and from strings.
`static final int`	**`MIN_SUPPLEMENTARY_CODE_P`**	The minimum value of a Unicode supplementary code point, constant `U+10000`.
`static final char`	**`MIN_SURROGATE`**	The minimum value of a Unicode surrogate code unit in the UTF-16 encoding, constant `'\uD800'`.
`static final char`	**`MIN_VALUE`**	The constant value of this field is the smallest value of type `char`, `'\u0000'`.
`static final byte`	**`MODIFIER_LETTER`**	General category "Lm" in the Unicode specification.
`static final byte`	**`MODIFIER_SYMBOL`**	General category "Sk" in the Unicode specification.
`static final byte`	**`NON_SPACING_MARK`**	

		General category "Mn" in the Unicode specification.
static final byte	**OTHER_LETTER**	General category "Lo" in the Unicode specification.
static final byte	**OTHER_NUMBER**	General category "No" in the Unicode specification.
static final byte	**OTHER_PUNCTUATION**	General category "Po" in the Unicode specification.
static final byte	**OTHER_SYMBOL**	General category "So" in the Unicode specification.
static final byte	**PARAGRAPH_SEPARATOR**	General category "Zp" in the Unicode specification.
static final byte	**PRIVATE_USE**	General category "Co" in the Unicode specification.
static final int	**SIZE**	The number of bits used to represent a `char` value in unsigned binary form, constant 16.
static final byte	**SPACE_SEPARATOR**	General category "Zs" in the Unicode specification.
static final byte	**START_PUNCTUATION**	General category "Ps" in the Unicode specification.
static final byte	**SURROGATE**	General category "Cs" in the Unicode specification.
static final byte	**TITLECASE_LETTER**	

		General category "Lt" in the Unicode specification.
static final Class<Character>	TYPE	The Class instance representing the primitive type char.
static final byte	UNASSIGNED	General category "Cn" in the Unicode specification.
static final byte	UPPERCASE_LETTER	General category "Lu" in the Unicode specification.

Constructor Summary

Constructors

Constructor	Description
Character(char value)	**Deprecated.** It is rarely appropriate to use this constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
static int	charCount (int codePoint)	Determines the number of char values needed to represent the specified character (Unicode code point).
char	charValue()	Returns the value of this Character object.
static int	codePointAt(char[] a, int index)	Returns the code point at the given index of the char array.
static int		

	codePointAt(char[] a, int index, int limit)	Returns the code point at the given index of the char array, where only array elements with index less than limit can be used.
static int	**codePointAt**(CharSequence seq, int index)	Returns the code point at the given index of the CharSequence.
static int	**codePointBefore**(char[] a, int index)	Returns the code point preceding the given index of the char array.
static int	**codePointBefore**(char[] a, int index, int start)	Returns the code point preceding the given index of the char array, where only array elements with index greater than or equal to start can be used.
static int	**codePointBefore**(CharSequence seq, int index)	Returns the code point preceding the given index of the CharSequence.
static int	**codePointCount**(char[] a, int offset, int count)	Returns the number of Unicode code points in a subarray of the char array argument.
static int	**codePointCount**(CharSequence seq, int beginIndex, int endIndex)	Returns the number of Unicode code points in the text range of the specified char sequence.
static int	**codePointOf**(String name)	Returns the code point value of the Unicode character specified by the given character name.
static int	**compare**(char x, char y)	Compares two char values numerically.

int	**compareTo**(Character anotherCharac	Compares two Character objects numerically.
Optional<DynamicConstantDesc·	**describeConstable**()	Returns an Optional containing the nominal descriptor for this instance.
static int	**digit**(char ch, int radix)	Returns the numeric value of the character ch in the specified radix.
static int	**digit**(int codePoint, int radix)	Returns the numeric value of the specified character (Unicode code point) in the specified radix.
boolean	**equals**(Object obj)	Compares this object against the specified object.
static char	**forDigit**(int digit, int radix)	Determines the character representation for a specific digit in the specified radix.
static byte	**getDirectionality**(char ch)	Returns the Unicode directionality property for the given character.
static byte	**getDirectionality**(int codePoint)	Returns the Unicode directionality property for the given character (Unicode code point).
static String	**getName**(int codePoint)	Returns the name of the specified character codePoint, or null if the code point is unassigned.
static int	**getNumericValue**(char ch)	Returns the int value that the specified

		Unicode character represents.
`static int`	**`getNumericValue`**`(int codePoint)`	Returns the `int` value that the specified character (Unicode code point) represents.
`static int`	**`getType`**`(char ch)`	Returns a value indicating a character's general category.
`static int`	**`getType`**`(int codePoint)`	Returns a value indicating a character's general category.
`int`	**`hashCode`**`()`	Returns a hash code for this `Character`; equal to the result of invoking `charValue()`.
`static int`	**`hashCode`**`(char value)`	Returns a hash code for a `char` value; compatible with `Character.hashCode()`.
`static char`	**`highSurrogate`**`(int codePoint)`	Returns the leading surrogate (a high surrogate code unit) of the surrogate pair representing the specified supplementary character (Unicode code point) in the UTF-16 encoding.
`static boolean`	**`isAlphabetic`**`(int codePoint)`	Determines if the specified character (Unicode code point) is alphabetic.
`static boolean`	**`isBmpCodePoint`**`(int codePoint)`	Determines whether the specified character (Unicode code point) is

		in the Basic Multilingual Plane (BMP).
static boolean	**isDefined**(char ch)	Determines if a character is defined in Unicode.
static boolean	**isDefined**(int codePoint)	Determines if a character (Unicode code point) is defined in Unicode.
static boolean	**isDigit**(char ch)	Determines if the specified character is a digit.
static boolean	**isDigit**(int codePoint)	Determines if the specified character (Unicode code point) is a digit.
static boolean	**isEmoji**(int codePoint)	Determines if the specified character (Unicode code point) is an Emoji.
static boolean	**isEmojiComponent**(int codePoint)	Determines if the specified character (Unicode code point) is an Emoji Component.
static boolean	**isEmojiModifier**(int codePoint)	Determines if the specified character (Unicode code point) is an Emoji Modifier.
static boolean	**isEmojiModifierBase**(int codePoint)	Determines if the specified character (Unicode code point) is an Emoji Modifier Base.
static boolean	**isEmojiPresentation**(int codePoint)	Determines if the specified character

		(Unicode code point) has the Emoji Presentation property by default.
static boolean	**isExtendedPictographic**(int codePoint)	Determines if the specified character (Unicode code point) is an Extended Pictographic.
static boolean	**isHighSurrogate**(char ch)	Determines if the given char value is a Unicode high-surrogate code unit (also known as *leading-surrogate code unit*).
static boolean	**isIdentifierIgnorable**(char ch)	Determines if the specified character should be regarded as an ignorable character in a Java identifier or a Unicode identifier.
static boolean	**isIdentifierIgnorable**(int codePoint)	Determines if the specified character (Unicode code point) should be regarded as an ignorable character in a Java identifier or a Unicode identifier.
static boolean	**isIdeographic**(int codePoint)	Determines if the specified character (Unicode code point) is a CJKV (Chinese, Japanese, Korean and Vietnamese) ideograph, as defined by the Unicode Standard.
static boolean	**isISOControl**(char ch)	

		Determines if the specified character is an ISO control character.
static boolean	**isISOControl**(int codePoint)	Determines if the referenced character (Unicode code point) is an ISO control character.
static boolean	**isJavaIdentifierPart**(char ch)	Determines if the specified character may be part of a Java identifier as other than the first character.
static boolean	**isJavaIdentifierPart**(int codePoint)	Determines if the character (Unicode code point) may be part of a Java identifier as other than the first character.
static boolean	**isJavaIdentifierStart**(char ch)	Determines if the specified character is permissible as the first character in a Java identifier.
static boolean	**isJavaIdentifierStart**(int codePoint)	Determines if the character (Unicode code point) is permissible as the first character in a Java identifier.
static boolean	**isJavaLetter**(char ch)	**Deprecated.** Replaced by isJavaIdentifierStart(char).
static boolean	**isJavaLetterOrDigit**(char ch)	**Deprecated.** Replaced by isJavaIdentifierPart(char).
static boolean	**isLetter**(char ch)	Determines if the specified character is a letter.
static boolean	**isLetter**(int codePoint)	

		Determines if the specified character (Unicode code point) is a letter.
static boolean	**isLetterOrDigit**(char ch)	Determines if the specified character is a letter or digit.
static boolean	**isLetterOrDigit**(int codePoint)	Determines if the specified character (Unicode code point) is a letter or digit.
static boolean	**isLowerCase**(char ch)	Determines if the specified character is a lowercase character.
static boolean	**isLowerCase**(int codePoint)	Determines if the specified character (Unicode code point) is a lowercase character.
static boolean	**isLowSurrogate**(char ch)	Determines if the given char value is a Unicode low-surrogate code unit (also known as *trailing-surrogate code unit*).
static boolean	**isMirrored**(char ch)	Determines whether the character is mirrored according to the Unicode specification.
static boolean	**isMirrored**(int codePoint)	Determines whether the specified character (Unicode code point) is mirrored according to the Unicode specification.
static boolean	**isSpace**(char ch)	**Deprecated.** Replaced by isWhitespace(char).
static boolean	**isSpaceChar**(char ch)	

		Determines if the specified character is a Unicode space character.
static boolean	**isSpaceChar**(int codePoint)	Determines if the specified character (Unicode code point) is a Unicode space character.
static boolean	**isSupplementaryCodePoint**(int codePoint)	Determines whether the specified character (Unicode code point) is in the supplementary character range.
static boolean	**isSurrogate**(char ch)	Determines if the given char value is a Unicode *surrogate code unit*.
static boolean	**isSurrogatePair**(char high, char low)	Determines whether the specified pair of char values is a valid Unicode surrogate pair.
static boolean	**isTitleCase**(char ch)	Determines if the specified character is a titlecase character.
static boolean	**isTitleCase**(int codePoint)	Determines if the specified character (Unicode code point) is a titlecase character.
static boolean	**isUnicodeIdentifierPart**(char ch)	Determines if the specified character may be part of a Unicode identifier as other than the first character.
static boolean	**isUnicodeIdentifierPart**(int codePoint)	Determines if the specified character (Unicode code point) may be part of a Unicode identifier as other than the first character.

static boolean	**isUnicodeIdentifierStart**(char ch)	Determines if the specified character is permissible as the first character in a Unicode identifier.
static boolean	**isUnicodeIdentifierStart**(int codePoint)	Determines if the specified character (Unicode code point) is permissible as the first character in a Unicode identifier.
static boolean	**isUpperCase**(char ch)	Determines if the specified character is an uppercase character.
static boolean	**isUpperCase**(int codePoint)	Determines if the specified character (Unicode code point) is an uppercase character.
static boolean	**isValidCodePoint**(int codePoint)	Determines whether the specified code point is a valid Unicode code point value.
static boolean	**isWhitespace**(char ch)	Determines if the specified character is white space according to Java.
static boolean	**isWhitespace**(int codePoint)	Determines if the specified character (Unicode code point) is white space according to Java.
static char	**lowSurrogate**(int codePoint)	Returns the trailing surrogate (a low surrogate code unit) of the surrogate pair representing the specified supplementary character (Unicode code

		point) in the UTF-16 encoding.
static int	**offsetByCodePoints**(char[] a, int start, int count, int index, int codePointOffset)	Returns the index within the given char subarray that is offset from the given index by codePointOffset code points.
static int	**offsetByCodePoints**(CharSequence seq, int index, int codePointOffset)	Returns the index within the given char sequence that is offset from the given index by codePointOffset code points.
static char	**reverseBytes**(char ch)	Returns the value obtained by reversing the order of the bytes in the specified char value.
static char[]	**toChars**(int codePoint)	Converts the specified character (Unicode code point) to its UTF-16 representation stored in a char array.
static int	**toChars**(int codePoint, char[] dst, int dstIndex)	Converts the specified character (Unicode code point) to its UTF-16 representation.
static int	**toCodePoint**(char high, char low)	Converts the specified surrogate pair to its supplementary code point value.
static char	**toLowerCase**(char ch)	

		Converts the character argument to lowercase using case mapping information from the UnicodeData file.
static int	**toLowerCase**(int codePoint)	Converts the character (Unicode code point) argument to lowercase using case mapping information from the UnicodeData file.
String	**toString**()	Returns a String object representing this Character's value.
static String	**toString**(char c)	Returns a String object representing the specified char.
static String	**toString**(int codePoint)	Returns a String object representing the specified character (Unicode code point).
static char	**toTitleCase**(char ch)	Converts the character argument to titlecase using case mapping information from the UnicodeData file.
static int	**toTitleCase**(int codePoint)	Converts the character (Unicode code point) argument to titlecase using case mapping information from the UnicodeData file.
static char	**toUpperCase**(char ch)	Converts the character argument to uppercase using case mapping information from the UnicodeData file.
static int		

	toUpperCase(int codePoint)	Converts the character (Unicode code point) argument to uppercase using case mapping information from the UnicodeData file.
static Character	**valueOf**(char c)	Returns a Character instance representing the specified char value.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

MIN_RADIX

public static final int MIN_RADIX

The minimum radix available for conversion to and from strings. The constant value of this field is the smallest value permitted for the radix argument in radix-conversion methods such as the digit method, the forDigit method, and the toString method of class Integer.

See Also:

digit(char, int),
forDigit(int, int),
Integer.toString(int, int),
Integer.valueOf(String),
Constant Field Values

MAX_RADIX

public static final int MAX_RADIX

The maximum radix available for conversion to and from strings. The constant value of this field is the largest value permitted for the radix argument in radix-conversion methods such as the digit method, the forDigit method, and the toString method of class Integer.

See Also:

digit(char, int),
forDigit(int, int),
Integer.toString(int, int),

`Integer.valueOf(String)`,
Constant Field Values

MIN_VALUE

```
public static final char MIN_VALUE
```

The constant value of this field is the smallest value of type `char`, `'\u0000'`.

Since:

1.0.2

See Also:

Constant Field Values

MAX_VALUE

```
public static final char MAX_VALUE
```

The constant value of this field is the largest value of type `char`, `'\uFFFF'`.

Since:

1.0.2

See Also:

Constant Field Values

TYPE

```
public static final Class<Character> TYPE
```

The `Class` instance representing the primitive type `char`.

Since:

1.1

UNASSIGNED

```
public static final byte UNASSIGNED
```

General category "Cn" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

UPPERCASE_LETTER

```
public static final byte UPPERCASE_LETTER
```

General category "Lu" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

LOWERCASE_LETTER

```
public static final byte LOWERCASE_LETTER
```

General category "Ll" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

TITLECASE_LETTER

```
public static final byte TITLECASE_LETTER
```

General category "Lt" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

MODIFIER_LETTER

```
public static final byte MODIFIER_LETTER
```

General category "Lm" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

OTHER_LETTER

```
public static final byte OTHER_LETTER
```

General category "Lo" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

NON_SPACING_MARK

```
public static final byte NON_SPACING_MARK
```

General category "Mn" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

ENCLOSING_MARK

```
public static final byte ENCLOSING_MARK
```

General category "Me" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

COMBINING_SPACING_MARK

```
public static final byte COMBINING_SPACING_MARK
```

General category "Mc" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

DECIMAL_DIGIT_NUMBER

```
public static final byte DECIMAL_DIGIT_NUMBER
```

General category "Nd" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

LETTER_NUMBER

```
public static final byte LETTER_NUMBER
```

General category "Nl" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

OTHER_NUMBER

```
public static final byte OTHER_NUMBER
```

General category "No" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

SPACE_SEPARATOR

```
public static final byte SPACE_SEPARATOR
```

General category "Zs" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

LINE_SEPARATOR

```
public static final byte LINE_SEPARATOR
```

General category "Zl" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

PARAGRAPH_SEPARATOR

```
public static final byte PARAGRAPH_SEPARATOR
```

General category "Zp" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

CONTROL

```
public static final byte CONTROL
```

General category "Cc" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

FORMAT

```
public static final byte FORMAT
```

General category "Cf" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

PRIVATE_USE

```
public static final byte PRIVATE_USE
```

General category "Co" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

SURROGATE

```
public static final byte SURROGATE
```

General category "Cs" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

DASH_PUNCTUATION

```
public static final byte DASH_PUNCTUATION
```

General category "Pd" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

START_PUNCTUATION

```
public static final byte START_PUNCTUATION
```

General category "Ps" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

END_PUNCTUATION

```
public static final byte END_PUNCTUATION
```

General category "Pe" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

CONNECTOR_PUNCTUATION

```
public static final byte CONNECTOR_PUNCTUATION
```

General category "Pc" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

OTHER_PUNCTUATION

`public static final byte OTHER_PUNCTUATION`

General category "Po" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

MATH_SYMBOL

`public static final byte MATH_SYMBOL`

General category "Sm" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

CURRENCY_SYMBOL

`public static final byte CURRENCY_SYMBOL`

General category "Sc" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

MODIFIER_SYMBOL

`public static final byte MODIFIER_SYMBOL`

General category "Sk" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

OTHER_SYMBOL

`public static final byte OTHER_SYMBOL`

General category "So" in the Unicode specification.

Since:

1.1

See Also:

Constant Field Values

INITIAL_QUOTE_PUNCTUATION

`public static final byte INITIAL_QUOTE_PUNCTUATION`

General category "Pi" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

FINAL_QUOTE_PUNCTUATION

`public static final byte FINAL_QUOTE_PUNCTUATION`

General category "Pf" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_UNDEFINED

`public static final byte DIRECTIONALITY_UNDEFINED`

Undefined bidirectional character type. Undefined `char` values have undefined directionality in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_LEFT_TO_RIGHT

```
public static final byte DIRECTIONALITY_LEFT_TO_RIGHT
```

Strong bidirectional character type "L" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_RIGHT_TO_LEFT

```
public static final byte DIRECTIONALITY_RIGHT_TO_LEFT
```

Strong bidirectional character type "R" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_RIGHT_TO_LEFT_ARABIC

```
public static final byte DIRECTIONALITY_RIGHT_TO_LEFT_ARABIC
```

Strong bidirectional character type "AL" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_EUROPEAN_NUMBER

```
public static final byte DIRECTIONALITY_EUROPEAN_NUMBER
```

Weak bidirectional character type "EN" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_EUROPEAN_NUMBER_SEPARATOR

```
public static final byte DIRECTIONALITY_EUROPEAN_NUMBER_SEPARATOR
```

Weak bidirectional character type "ES" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_EUROPEAN_NUMBER_TERMINATOR

```
public static final byte DIRECTIONALITY_EUROPEAN_NUMBER_TERMINATOR
```

Weak bidirectional character type "ET" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_ARABIC_NUMBER

```
public static final byte DIRECTIONALITY_ARABIC_NUMBER
```

Weak bidirectional character type "AN" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_COMMON_NUMBER_SEPARATOR

```
public static final byte DIRECTIONALITY_COMMON_NUMBER_SEPARATOR
```

Weak bidirectional character type "CS" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_NONSPACING_MARK

```
public static final byte DIRECTIONALITY_NONSPACING_MARK
```

Weak bidirectional character type "NSM" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_BOUNDARY_NEUTRAL

```
public static final byte DIRECTIONALITY_BOUNDARY_NEUTRAL
```

Weak bidirectional character type "BN" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_PARAGRAPH_SEPARATOR

```
public static final byte DIRECTIONALITY_PARAGRAPH_SEPARATOR
```

Neutral bidirectional character type "B" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_SEGMENT_SEPARATOR

```
public static final byte DIRECTIONALITY_SEGMENT_SEPARATOR
```

Neutral bidirectional character type "S" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_WHITESPACE

```
public static final byte DIRECTIONALITY_WHITESPACE
```

Neutral bidirectional character type "WS" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_OTHER_NEUTRALS

```
public static final byte DIRECTIONALITY_OTHER_NEUTRALS
```

Neutral bidirectional character type "ON" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_LEFT_TO_RIGHT_EMBEDDING

```
public static final byte DIRECTIONALITY_LEFT_TO_RIGHT_EMBEDDING
```

Strong bidirectional character type "LRE" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_LEFT_TO_RIGHT_OVERRIDE

```
public static final byte DIRECTIONALITY_LEFT_TO_RIGHT_OVERRIDE
```

Strong bidirectional character type "LRO" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_RIGHT_TO_LEFT_EMBEDDING

```
public static final byte DIRECTIONALITY_RIGHT_TO_LEFT_EMBEDDING
```

Strong bidirectional character type "RLE" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_RIGHT_TO_LEFT_OVERRIDE

```
public static final byte DIRECTIONALITY_RIGHT_TO_LEFT_OVERRIDE
```

Strong bidirectional character type "RLO" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_POP_DIRECTIONAL_FORMAT

```
public static final byte DIRECTIONALITY_POP_DIRECTIONAL_FORMAT
```

Weak bidirectional character type "PDF" in the Unicode specification.

Since:

1.4

See Also:

Constant Field Values

DIRECTIONALITY_LEFT_TO_RIGHT_ISOLATE

```
public static final byte DIRECTIONALITY_LEFT_TO_RIGHT_ISOLATE
```

Weak bidirectional character type "LRI" in the Unicode specification.

Since:

9

See Also:

Constant Field Values

DIRECTIONALITY_RIGHT_TO_LEFT_ISOLATE

```
public static final byte DIRECTIONALITY_RIGHT_TO_LEFT_ISOLATE
```

Weak bidirectional character type "RLI" in the Unicode specification.

Since:

9

See Also:

Constant Field Values

DIRECTIONALITY_FIRST_STRONG_ISOLATE

```
public static final byte DIRECTIONALITY_FIRST_STRONG_ISOLATE
```

Weak bidirectional character type "FSI" in the Unicode specification.

Since:

9

See Also:

Constant Field Values

DIRECTIONALITY_POP_DIRECTIONAL_ISOLATE

```
public static final byte DIRECTIONALITY_POP_DIRECTIONAL_ISOLATE
```

Weak bidirectional character type "PDI" in the Unicode specification.

Since:

9

See Also:

Constant Field Values

MIN_HIGH_SURROGATE

```
public static final char MIN_HIGH_SURROGATE
```

The minimum value of a Unicode high-surrogate code unit in the UTF-16 encoding, constant `'\uD800'`. A high-surrogate is also known as a *leading-surrogate*.

Since:

1.5

See Also:

Constant Field Values

MAX_HIGH_SURROGATE

```
public static final char MAX_HIGH_SURROGATE
```

The maximum value of a Unicode high-surrogate code unit in the UTF-16 encoding, constant `'\uDBFF'`. A high-surrogate is also known as a *leading-surrogate*.

Since:

1.5

See Also:

Constant Field Values

MIN_LOW_SURROGATE

```
public static final char MIN_LOW_SURROGATE
```

The minimum value of a Unicode low-surrogate code unit in the UTF-16 encoding, constant `'\uDC00'`. A low-surrogate is also known as a *trailing-surrogate*.

Since:

1.5

See Also:

Constant Field Values

MAX_LOW_SURROGATE

```
public static final char MAX_LOW_SURROGATE
```

The maximum value of a Unicode low-surrogate code unit in the UTF-16 encoding, constant `'\uDFFF'`. A low-surrogate is also known as a *trailing-surrogate*.

Since:

1.5

See Also:

Constant Field Values

MIN_SURROGATE

```
public static final char MIN_SURROGATE
```

The minimum value of a Unicode surrogate code unit in the UTF-16 encoding, constant `'\uD800'`.

Since:

1.5

See Also:

Constant Field Values

MAX_SURROGATE

```
public static final char MAX_SURROGATE
```

The maximum value of a Unicode surrogate code unit in the UTF-16 encoding, constant `'\uDFFF'`.

Since:

1.5

See Also:

Constant Field Values

MIN_SUPPLEMENTARY_CODE_POINT

```
public static final int MIN_SUPPLEMENTARY_CODE_POINT
```

The minimum value of a Unicode supplementary code point, constant `U+10000`.

Since:

1.5

See Also:

Constant Field Values

MIN_CODE_POINT

```
public static final int MIN_CODE_POINT
```

The minimum value of a Unicode code point, constant `U+0000`.

Since:

1.5

See Also:

Constant Field Values

MAX_CODE_POINT

```
public static final int MAX_CODE_POINT
```

The maximum value of a Unicode code point, constant `U+10FFFF`.

Since:

1.5

See Also:

Constant Field Values

SIZE

```
public static final int SIZE
```

The number of bits used to represent a `char` value in unsigned binary form, constant 16.

Since:

1.5

See Also:

Constant Field Values

BYTES

```
public static final int BYTES
```

The number of bytes used to represent a `char` value in unsigned binary form.

Since:

1.8

See Also:

Constant Field Values

Constructor Details

Character

```
@Deprecated(since="9")
public Character(char value)
```

> **Deprecated.**
> *It is rarely appropriate to use this constructor. The static factory `valueOf(char)` is generally a better choice, as it is likely to yield significantly better space and time performance.*

Constructs a newly allocated `Character` object that represents the specified `char` value.

Parameters:

`value` - the value to be represented by the `Character` object.

Method Details

describeConstable

```
public Optional<DynamicConstantDesc<Character>> describeConstable()
```

Returns an `Optional` containing the nominal descriptor for this instance.

Specified by:

`describeConstable` in interface `Constable`

Returns:

an `Optional` describing the Character instance

Since:

15

valueOf

```
public static Character valueOf(char c)
```

Returns a `Character` instance representing the specified `char` value. If a new `Character` instance is not required, this method should generally be used in preference to the constructor `Character(char)`, as this method is likely to yield significantly better space and time performance by caching frequently requested values. This method will always cache values in the range `'\u0000'` to `'\u007F'`, inclusive, and may cache other values outside of this range.

Parameters:

`c` - a char value.

Returns:

a `Character` instance representing `c`.

Since:

1.5

charValue

```
public char charValue()
```

Returns the value of this `Character` object.

Returns:

the primitive `char` value represented by this object.

hashCode

```
public int hashCode()
```

Returns a hash code for this `Character`; equal to the result of invoking `charValue()`.

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this `Character`

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(Object)`

hashCode

```
public static int hashCode(char value)
```

Returns a hash code for a `char` value; compatible with `Character.hashCode()`.

Parameters:

`value` - The `char` for which to return a hash code.

Returns:

a hash code value for a `char` value.

Since:

1.8

equals

```
public boolean equals(Object obj)
```

Compares this object against the specified object. The result is `true` if and only if the argument is not `null` and is a `Character` object that represents the same `char` value as this object.

Overrides:

`equals` in class `Object`

Parameters:

`obj` - the object to compare with.

Returns:

`true` if the objects are the same; `false` otherwise.

See Also:

`Object.hashCode()`, `HashMap`

toString

```
public String toString()
```

Returns a `String` object representing this `Character`'s value. The result is a string of length 1 whose sole component is the primitive `char` value represented by this `Character` object.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this object.

toString

```
public static String toString(char c)
```

Returns a `String` object representing the specified `char`. The result is a string of length 1 consisting solely of the specified `char`.

API Note:

This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `toString(int)` method.

Parameters:

`c` - the `char` to be converted

Returns:

the string representation of the specified `char`

Since:

1.4

toString

```
public static String toString(int codePoint)
```

Returns a `String` object representing the specified character (Unicode code point). The result is a string of length 1 or 2, consisting solely of the specified `codePoint`.

Parameters:

`codePoint` - the `codePoint` to be converted

Returns:

the string representation of the specified `codePoint`

Throws:

`IllegalArgumentException` - if the specified `codePoint` is not a valid Unicode code point.

Since:

11

isValidCodePoint

```
public static boolean isValidCodePoint(int codePoint)
```

Determines whether the specified code point is a valid Unicode code point value.

Parameters:

`codePoint` - the Unicode code point to be tested

Returns:

`true` if the specified code point value is between `MIN_CODE_POINT` and `MAX_CODE_POINT` inclusive; `false` otherwise.

Since:

1.5

isBmpCodePoint

```
public static boolean isBmpCodePoint(int codePoint)
```

Determines whether the specified character (Unicode code point) is in the Basic Multilingual Plane (BMP). Such code points can be represented using a single `char`.

Parameters:

`codePoint` - the character (Unicode code point) to be tested

Returns:

`true` if the specified code point is between `MIN_VALUE` and `MAX_VALUE` inclusive; `false` otherwise.

Since:

1.7

isSupplementaryCodePoint

```
public static boolean isSupplementaryCodePoint(int codePoint)
```

Determines whether the specified character (Unicode code point) is in the supplementary character range.

Parameters:

`codePoint` - the character (Unicode code point) to be tested

Returns:

`true` if the specified code point is between `MIN_SUPPLEMENTARY_CODE_POINT` and `MAX_CODE_POINT` inclusive; `false` otherwise.

Since:

1.5

isHighSurrogate

```
public static boolean isHighSurrogate(char ch)
```

Determines if the given `char` value is a Unicode high-surrogate code unit (also known as *leading-surrogate code unit*).

Such values do not represent characters by themselves, but are used in the representation of supplementary characters in the UTF-16 encoding.

Parameters:

`ch` - the `char` value to be tested.

Returns:

`true` if the `char` value is between `MIN_HIGH_SURROGATE` and `MAX_HIGH_SURROGATE` inclusive; `false` otherwise.

Since:

1.5

See Also:

`isLowSurrogate(char)`, `Character.UnicodeBlock.of(int)`

isLowSurrogate

```
public static boolean isLowSurrogate(char ch)
```

Determines if the given `char` value is a Unicode low-surrogate code unit (also known as *trailing-surrogate code unit*).

Such values do not represent characters by themselves, but are used in the representation of supplementary characters in the UTF-16 encoding.

Parameters:

`ch` - the `char` value to be tested.

Returns:

`true` if the `char` value is between `MIN_LOW_SURROGATE` and `MAX_LOW_SURROGATE` inclusive; `false` otherwise.

Since:

1.5

See Also:

`isHighSurrogate(char)`

isSurrogate

```
public static boolean isSurrogate(char ch)
```

Determines if the given `char` value is a Unicode *surrogate code unit*.

Such values do not represent characters by themselves, but are used in the representation of supplementary characters in the UTF-16 encoding.

A char value is a surrogate code unit if and only if it is either a low-surrogate code unit or a high-surrogate code unit.

Parameters:

`ch` - the `char` value to be tested.

Returns:

`true` if the `char` value is between `MIN_SURROGATE` and `MAX_SURROGATE` inclusive; `false` otherwise.

Since:

1.7

isSurrogatePair

```
public static boolean isSurrogatePair(char high,
                                      char low)
```

Determines whether the specified pair of `char` values is a valid Unicode surrogate pair.

This method is equivalent to the expression:

```
isHighSurrogate(high) && isLowSurrogate(low)
```

Parameters:

`high` - the high-surrogate code value to be tested

`low` - the low-surrogate code value to be tested

Returns:

`true` if the specified high and low-surrogate code values represent a valid surrogate pair; `false` otherwise.

Since:

1.5

charCount

```
public static int charCount(int codePoint)
```

Determines the number of `char` values needed to represent the specified character (Unicode code point). If the specified character is equal to or greater than 0x10000, then the method returns 2. Otherwise, the method returns 1.

This method doesn't validate the specified character to be a valid Unicode code point. The caller must validate the character value using `isValidCodePoint` if necessary.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

2 if the character is a valid supplementary character; 1 otherwise.

Since:

1.5

See Also:

`isSupplementaryCodePoint(int)`

toCodePoint

```
public static int toCodePoint(char high,
                              char low)
```

Converts the specified surrogate pair to its supplementary code point value. This method does not validate the specified surrogate pair. The caller must validate it using `isSurrogatePair` if necessary.

Parameters:

`high` - the high-surrogate code unit

`low` - the low-surrogate code unit

Returns:

the supplementary code point composed from the specified surrogate pair.

Since:

1.5

codePointAt

```
public static int codePointAt(CharSequence seq,
                              int index)
```

Returns the code point at the given index of the `CharSequence`. If the `char` value at the given index in the `CharSequence` is in the high-surrogate range, the following index is less than the length of the `CharSequence`, and the `char` value at the following index is in the low-surrogate range, then the supplementary code point corresponding to this surrogate pair is returned. Otherwise, the `char` value at the given index is returned.

Parameters:

`seq` - a sequence of `char` values (Unicode code units)

`index` - the index to the `char` values (Unicode code units) in `seq` to be converted

Returns:

the Unicode code point at the given index

Throws:

`NullPointerException` - if `seq` is null.

`IndexOutOfBoundsException` - if the value `index` is negative or not less than `seq.length()`.

Since:

1.5

codePointAt

```
public static int codePointAt(char[] a,
                              int index)
```

Returns the code point at the given index of the `char` array. If the `char` value at the given index in the `char` array is in the high-surrogate range, the following index is less than the length of the `char` array, and the `char` value at the following index is in the low-surrogate range, then the supplementary code point corresponding to this surrogate pair is returned. Otherwise, the `char` value at the given index is returned.

Parameters:

`a` - the `char` array

`index` - the index to the `char` values (Unicode code units) in the `char` array to be converted

Returns:

the Unicode code point at the given index

Throws:

`NullPointerException` - if `a` is null.

`IndexOutOfBoundsException` - if the value `index` is negative or not less than the length of the `char` array.

Since:

1.5

codePointAt

```
public static int codePointAt(char[] a,
                              int index,
                              int limit)
```

Returns the code point at the given index of the `char` array, where only array elements with `index` less than `limit` can be used. If the `char` value at the given index in the `char` array is in the high-surrogate range, the following index is less than the `limit`, and the `char` value at the following index is in the low-surrogate range, then the supplementary code point corresponding to this surrogate pair is returned. Otherwise, the `char` value at the given index is returned.

Parameters:

`a` - the `char` array

`index` - the index to the `char` values (Unicode code units) in the `char` array to be converted

`limit` - the index after the last array element that can be used in the `char` array

Returns:

the Unicode code point at the given index

Throws:

`NullPointerException` - if `a` is null.

`IndexOutOfBoundsException` - if the `index` argument is negative or not less than the `limit` argument, or if the `limit` argument is negative or greater than the length of the `char` array.

Since:

1.5

codePointBefore

```
public static int codePointBefore(CharSequence seq,
                                  int index)
```

Returns the code point preceding the given index of the `CharSequence`. If the `char` value at `(index - 1)` in the `CharSequence` is in the low-surrogate range, `(index - 2)` is not negative, and the `char` value at `(index - 2)` in the `CharSequence` is in the high-surrogate range, then the supplementary code point corresponding to this surrogate pair is returned. Otherwise, the `char` value at `(index - 1)` is returned.

Parameters:

`seq` - the `CharSequence` instance

`index` - the index following the code point that should be returned

Returns:

the Unicode code point value before the given index.

Throws:

`NullPointerException` - if `seq` is null.

`IndexOutOfBoundsException` - if the `index` argument is less than 1 or greater than `seq.length()`.

Since:

1.5

codePointBefore

```
public static int codePointBefore(char[] a,
                                  int index)
```

Returns the code point preceding the given index of the `char` array. If the `char` value at `(index - 1)` in the `char` array is in the low-surrogate range, `(index - 2)` is not negative, and the `char` value at `(index - 2)` in the `char` array is in the high-surrogate range, then the supplementary code point corresponding to this surrogate pair is returned. Otherwise, the `char` value at `(index - 1)` is returned.

Parameters:

`a` - the `char` array

`index` - the index following the code point that should be returned

Returns:

the Unicode code point value before the given index.

Throws:

`NullPointerException` - if `a` is null.

`IndexOutOfBoundsException` - if the `index` argument is less than 1 or greater than the length of the `char` array

Since:

1.5

codePointBefore

```
public static int codePointBefore(char[] a,
                                  int index,
                                  int start)
```

Returns the code point preceding the given index of the `char` array, where only array elements with `index` greater than or equal to `start` can be used. If the `char` value at `(index - 1)` in the `char` array is in the low-surrogate range, `(index - 2)` is not less than `start`, and the `char` value at `(index - 2)` in the `char` array is in the high-surrogate range, then the supplementary code point corresponding to this surrogate pair is returned. Otherwise, the `char` value at `(index - 1)` is returned.

Parameters:

`a` - the `char` array

`index` - the index following the code point that should be returned

`start` - the index of the first array element in the `char` array

Returns:

the Unicode code point value before the given index.

Throws:

`NullPointerException` - if `a` is null.

`IndexOutOfBoundsException` - if the `index` argument is not greater than the `start` argument or is greater than the length of the `char` array, or if the `start` argument is negative or not less than the length of the `char` array.

Since:

1.5

highSurrogate

```
public static char highSurrogate(int codePoint)
```

Returns the leading surrogate (a high surrogate code unit) of the surrogate pair representing the specified supplementary character (Unicode code point) in the UTF-16 encoding. If the specified character is not a supplementary character, an unspecified `char` is returned.

If `isSupplementaryCodePoint(x)` is `true`, then `isHighSurrogate(highSurrogate(x))` and `toCodePoint(highSurrogate(x), lowSurrogate(x)) == x` are also always `true`.

Parameters:

`codePoint` - a supplementary character (Unicode code point)

Returns:

the leading surrogate code unit used to represent the character in the UTF-16 encoding

Since:

1.7

lowSurrogate

```
public static char lowSurrogate(int codePoint)
```

Returns the trailing surrogate (a low surrogate code unit) of the surrogate pair representing the specified supplementary character (Unicode code point) in the UTF-16 encoding. If the specified character is not a supplementary character, an unspecified `char` is returned.

If `isSupplementaryCodePoint(x)` is `true`, then `isLowSurrogate(lowSurrogate(x))` and `toCodePoint(highSurrogate(x), lowSurrogate(x)) == x` are also always `true`.

Parameters:

`codePoint` - a supplementary character (Unicode code point)

Returns:

the trailing surrogate code unit used to represent the character in the UTF-16 encoding

Since:

1.7

toChars

```
public static int toChars(int codePoint,
                          char[] dst,
                          int dstIndex)
```

Converts the specified character (Unicode code point) to its UTF-16 representation. If the specified code point is a BMP (Basic Multilingual Plane or Plane 0) value, the same value is stored in `dst[dstIndex]`, and 1 is returned. If the specified code point is a supplementary character, its surrogate values are stored in `dst[dstIndex]` (high-surrogate) and `dst[dstIndex+1]` (low-surrogate), and 2 is returned.

Parameters:

`codePoint` - the character (Unicode code point) to be converted.

`dst` - an array of `char` in which the `codePoint`'s UTF-16 value is stored.

`dstIndex` - the start index into the `dst` array where the converted value is stored.

Returns:

1 if the code point is a BMP code point, 2 if the code point is a supplementary code point.

Throws:

`IllegalArgumentException` - if the specified `codePoint` is not a valid Unicode code point.

`NullPointerException` - if the specified `dst` is null.

`IndexOutOfBoundsException` - if `dstIndex` is negative or not less than `dst.length`, or if `dst` at `dstIndex` doesn't have enough array element(s) to store the resulting `char` value(s). (If `dstIndex` is equal to `dst.length-1` and the specified `codePoint` is a supplementary character, the high-surrogate value is not stored in `dst[dstIndex]`.)

Since:

1.5

toChars

```
public static char[] toChars(int codePoint)
```

Converts the specified character (Unicode code point) to its UTF-16 representation stored in a `char` array. If the specified code point is a BMP (Basic Multilingual Plane or Plane 0) value, the resulting `char` array has the same value as `codePoint`. If the specified code point is a supplementary code point, the resulting `char` array has the corresponding surrogate pair.

Parameters:

`codePoint` - a Unicode code point

Returns:

a `char` array having `codePoint`'s UTF-16 representation.

Throws:

`IllegalArgumentException` - if the specified `codePoint` is not a valid Unicode code point.

Since:

1.5

codePointCount

```
public static int codePointCount(CharSequence seq,
                                 int beginIndex,
                                 int endIndex)
```

Returns the number of Unicode code points in the text range of the specified char sequence. The text range begins at the specified `beginIndex` and extends to the `char` at index `endIndex - 1`. Thus the length (in `char`s) of the text range is `endIndex-beginIndex`. Unpaired surrogates within the text range count as one code point each.

Parameters:

`seq` - the char sequence

`beginIndex` - the index to the first `char` of the text range.

`endIndex` - the index after the last `char` of the text range.

Returns:

the number of Unicode code points in the specified text range

Throws:

`NullPointerException` - if `seq` is null.

`IndexOutOfBoundsException` - if the `beginIndex` is negative, or `endIndex` is larger than the length of the given sequence, or `beginIndex` is larger than `endIndex`.

Since:

1.5

codePointCount

```
public static int codePointCount(char[] a,
                                 int offset,
                                 int count)
```

Returns the number of Unicode code points in a subarray of the `char` array argument. The `offset` argument is the index of the first `char` of the subarray and the `count` argument specifies the length of the subarray in `chars`. Unpaired surrogates within the subarray count as one code point each.

Parameters:

`a` - the `char` array

`offset` - the index of the first `char` in the given `char` array

`count` - the length of the subarray in `chars`

Returns:

the number of Unicode code points in the specified subarray

Throws:

`NullPointerException` - if `a` is null.

`IndexOutOfBoundsException` - if `offset` or `count` is negative, or if `offset + count` is larger than the length of the given array.

Since:

1.5

offsetByCodePoints

```
public static int offsetByCodePoints(CharSequence seq,
                                     int index,
                                     int codePointOffset)
```

Returns
the index within the given char sequence that is offset from the given `index` by `codePointOffset` code points. Unpaired surrogates within the text range given by `index` and `codePointOffset` count as one code point each.

Parameters:

`seq` - the char sequence

`index` - the index to be offset

`codePointOffset` - the offset in code points

Returns:

the index within the char sequence

Throws:

`NullPointerException` - if `seq` is null.

`IndexOutOfBoundsException` - if `index` is negative or larger than the length of the char sequence, or if `codePointOffset` is positive and the subsequence starting with `index` has fewer than `codePointOffset` code points, or if `codePointOffset` is negative and the subsequence before `index` has fewer than the absolute value of `codePointOffset` code points.

Since:

1.5

offsetByCodePoints

```
public static int offsetByCodePoints(char[] a,
                                      int start,
                                      int count,
                                      int index,
                                      int codePointOffset)
```

Returns the index within the given `char` subarray that is offset from the given `index` by `codePointOffset` code points. The `start` and `count` arguments specify a subarray of the `char` array. Unpaired surrogates within the text range given by `index` and `codePointOffset` count as one code point each.

Parameters:

`a` - the `char` array

`start` - the index of the first `char` of the subarray

`count` - the length of the subarray in `char`s

`index` - the index to be offset

`codePointOffset` - the offset in code points

Returns:

the index within the subarray

Throws:

`NullPointerException` - if `a` is null.

`IndexOutOfBoundsException` - if `start` or `count` is negative, or if `start + count` is larger than the length of the given array, or if `index` is less than `start` or larger then `start + count`, or if `codePointOffset` is positive and the text range starting with `index` and ending with `start + count - 1` has fewer than `codePointOffset` code points, or if `codePointOffset` is negative and the text range starting with `start` and ending with `index - 1` has fewer than the absolute value of `codePointOffset` code points.

Since:

1.5

isLowerCase

```
public static boolean isLowerCase(char ch)
```

Determines if the specified character is a lowercase character.

A character is lowercase if its general category type, provided by `Character.getType(ch)`, is `LOWERCASE_LETTER`, or it has contributory property Other_Lowercase as defined by the Unicode Standard.

The following are examples of lowercase characters:

```
a b c d e f g h i j k l m n o p q r s t u v w x y z
'\u00DF' '\u00E0' '\u00E1' '\u00E2' '\u00E3' '\u00E4' '\u00E5' '\u00E6'
'\u00E7' '\u00E8' '\u00E9' '\u00EA' '\u00EB' '\u00EC' '\u00ED' '\u00EE'
'\u00EF' '\u00F0' '\u00F1' '\u00F2' '\u00F3' '\u00F4' '\u00F5' '\u00F6'
'\u00F8' '\u00F9' '\u00FA' '\u00FB' '\u00FC' '\u00FD' '\u00FE' '\u00FF'
```

Many other Unicode characters are lowercase too.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isLowerCase(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character is lowercase; `false` otherwise.

See Also:

`isLowerCase(char)`, `isTitleCase(char)`, `toLowerCase(char)`, `getType(char)`

isLowerCase

```
public static boolean isLowerCase(int codePoint)
```

Determines if the specified character (Unicode code point) is a lowercase character.

A character is lowercase if its general category type, provided by `getType(codePoint)`, is `LOWERCASE_LETTER`, or it has contributory property Other_Lowercase as defined by the Unicode Standard.

The following are examples of lowercase characters:

```
a b c d e f g h i j k l m n o p q r s t u v w x y z
'\u00DF' '\u00E0' '\u00E1' '\u00E2' '\u00E3' '\u00E4' '\u00E5' '\u00E6'
'\u00E7' '\u00E8' '\u00E9' '\u00EA' '\u00EB' '\u00EC' '\u00ED' '\u00EE'
'\u00EF' '\u00F0' '\u00F1' '\u00F2' '\u00F3' '\u00F4' '\u00F5' '\u00F6'
'\u00F8' '\u00F9' '\u00FA' '\u00FB' '\u00FC' '\u00FD' '\u00FE' '\u00FF'
```

Many other Unicode characters are lowercase too.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is lowercase; `false` otherwise.

Since:

1.5

See Also:

`isLowerCase(int)`, `isTitleCase(int)`, `toLowerCase(int)`, `getType(int)`

isUpperCase

```
public static boolean isUpperCase(char ch)
```

Determines if the specified character is an uppercase character.

A character is uppercase if its general category type, provided by `Character.getType(ch)`, is `UPPERCASE_LETTER`. or it has contributory property Other_Uppercase as defined by the Unicode Standard.

The following are examples of uppercase characters:

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
'\u00C0' '\u00C1' '\u00C2' '\u00C3' '\u00C4' '\u00C5' '\u00C6' '\u00C7'
'\u00C8' '\u00C9' '\u00CA' '\u00CB' '\u00CC' '\u00CD' '\u00CE' '\u00CF'
'\u00D0' '\u00D1' '\u00D2' '\u00D3' '\u00D4' '\u00D5' '\u00D6' '\u00D8'
'\u00D9' '\u00DA' '\u00DB' '\u00DC' '\u00DD' '\u00DE'
```

Many other Unicode characters are uppercase too.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isUpperCase(int)` method.

Parameters:

ch - the character to be tested.

Returns:

true if the character is uppercase; false otherwise.

Since:

1.0

See Also:

isLowerCase(char), isTitleCase(char), toUpperCase(char), getType(char)

isUpperCase

```
public static boolean isUpperCase(int codePoint)
```

Determines if the specified character (Unicode code point) is an uppercase character.

A character is uppercase if its general category type, provided by getType(codePoint), is UPPERCASE_LETTER, or it has contributory property Other_Uppercase as defined by the Unicode Standard.

The following are examples of uppercase characters:

```
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
'\u00C0' '\u00C1' '\u00C2' '\u00C3' '\u00C4' '\u00C5' '\u00C6' '\u00C7'
'\u00C8' '\u00C9' '\u00CA' '\u00CB' '\u00CC' '\u00CD' '\u00CE' '\u00CF'
'\u00D0' '\u00D1' '\u00D2' '\u00D3' '\u00D4' '\u00D5' '\u00D6' '\u00D8'
'\u00D9' '\u00DA' '\u00DB' '\u00DC' '\u00DD' '\u00DE'
```

Many other Unicode characters are uppercase too.

Parameters:

codePoint - the character (Unicode code point) to be tested.

Returns:

true if the character is uppercase; false otherwise.

Since:

1.5

See Also:

isLowerCase(int), isTitleCase(int), toUpperCase(int), getType(int)

isTitleCase

```
public static boolean isTitleCase(char ch)
```

Determines if the specified character is a titlecase character.

A character is a titlecase character if its general category type, provided by `Character.getType(ch)`, is `TITLECASE_LETTER`.

Some characters look like pairs of Latin letters. For example, there is an uppercase letter that looks like "LJ" and has a corresponding lowercase letter that looks like "lj". A third form, which looks like "Lj", is the appropriate form to use when rendering a word in lowercase with initial capitals, as for a book title.

These are some of the Unicode characters for which this method returns `true`:

- `LATIN CAPITAL LETTER D WITH SMALL LETTER Z WITH CARON`
- `LATIN CAPITAL LETTER L WITH SMALL LETTER J`
- `LATIN CAPITAL LETTER N WITH SMALL LETTER J`
- `LATIN CAPITAL LETTER D WITH SMALL LETTER Z`

Many other Unicode characters are titlecase too.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isTitleCase(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character is titlecase; `false` otherwise.

Since:

1.0.2

See Also:

`isLowerCase(char)`, `isUpperCase(char)`, `toTitleCase(char)`, `getType(char)`

isTitleCase

```
public static boolean isTitleCase(int codePoint)
```

Determines if the specified character (Unicode code point) is a titlecase character.

A character is a titlecase character if its general category type, provided by `getType(codePoint)`, is `TITLECASE_LETTER`.

Some characters look like pairs of Latin letters. For example, there is an uppercase letter that looks like "LJ" and has a corresponding lowercase letter

that looks like "lj". A third form, which looks like "Lj", is the appropriate form to use when rendering a word in lowercase with initial capitals, as for a book title.

These are some of the Unicode characters for which this method returns `true`:

- `LATIN CAPITAL LETTER D WITH SMALL LETTER Z WITH CARON`
- `LATIN CAPITAL LETTER L WITH SMALL LETTER J`
- `LATIN CAPITAL LETTER N WITH SMALL LETTER J`
- `LATIN CAPITAL LETTER D WITH SMALL LETTER Z`

Many other Unicode characters are titlecase too.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is titlecase; `false` otherwise.

Since:

1.5

See Also:

`isLowerCase(int)`, `isUpperCase(int)`, `toTitleCase(int)`, `getType(int)`

isDigit

```
public static boolean isDigit(char ch)
```

Determines if the specified character is a digit.

A character is a digit if its general category type, provided by `Character.getType(ch)`, is `DECIMAL_DIGIT_NUMBER`.

Some Unicode character ranges that contain digits:

- `'\u0030'` through `'\u0039'`, ISO-LATIN-1 digits (`'0'` through `'9'`)
- `'\u0660'` through `'\u0669'`, Arabic-Indic digits
- `'\u06F0'` through `'\u06F9'`, Extended Arabic-Indic digits
- `'\u0966'` through `'\u096F'`, Devanagari digits
- `'\uFF10'` through `'\uFF19'`, Fullwidth digits

Many other character ranges contain digits as well.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isDigit(int)` method.

Parameters:

ch - the character to be tested.

Returns:

true if the character is a digit; false otherwise.

See Also:

digit(char, int),
forDigit(int, int),
getType(char)

isDigit

```
public static boolean isDigit(int codePoint)
```

Determines if the specified character (Unicode code point) is a digit.

A character is a digit if its general category type, provided by getType(codePoint), is DECIMAL_DIGIT_NUMBER.

Some Unicode character ranges that contain digits:

- '\u0030' through '\u0039', ISO-LATIN-1 digits ('0' through '9')
- '\u0660' through '\u0669', Arabic-Indic digits
- '\u06F0' through '\u06F9', Extended Arabic-Indic digits
- '\u0966' through '\u096F', Devanagari digits
- '\uFF10' through '\uFF19', Fullwidth digits

Many other character ranges contain digits as well.

Parameters:

codePoint - the character (Unicode code point) to be tested.

Returns:

true if the character is a digit; false otherwise.

Since:

1.5

See Also:

forDigit(int, int),
getType(int)

isDefined

```
public static boolean isDefined(char ch)
```

Determines if a character is defined in Unicode.

A character is defined if at least one of the following is true:

- It has an entry in the UnicodeData file.

- It has a value in a range defined by the UnicodeData file.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isDefined(int)` method.

Parameters:

`ch` - the character to be tested

Returns:

`true` if the character has a defined meaning in Unicode; `false` otherwise.

Since:

1.0.2

See Also:

`isDigit(char)`, `isLetter(char)`, `isLetterOrDigit(char)`, `isLowerCase(char)`, `isTitleCase(char)`, `isUpperCase(char)`

isDefined

```
public static boolean isDefined(int codePoint)
```

Determines if a character (Unicode code point) is defined in Unicode.

A character is defined if at least one of the following is true:

- It has an entry in the UnicodeData file.
- It has a value in a range defined by the UnicodeData file.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character has a defined meaning in Unicode; `false` otherwise.

Since:

1.5

See Also:

`isDigit(int)`, `isLetter(int)`, `isLetterOrDigit(int)`, `isLowerCase(int)`, `isTitleCase(int)`, `isUpperCase(int)`

isLetter

```
public static boolean isLetter(char ch)
```

Determines if the specified character is a letter.

A character is considered to be a letter if its general category type, provided by `Character.getType(ch)`, is any of the following:

- `UPPERCASE_LETTER`
- `LOWERCASE_LETTER`
- `TITLECASE_LETTER`
- `MODIFIER_LETTER`
- `OTHER_LETTER`

Not all letters have case. Many characters are letters but are neither uppercase nor lowercase nor titlecase.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isLetter(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character is a letter; `false` otherwise.

See Also:

`isDigit(char)`, `isJavaIdentifierStart(char)`, `isJavaLetter(char)`, `isJavaLetterOrDigit(char)`, `isLetterOrDigit(char)`, `isLowerCase(char)`, `isTitleCase(char)`, `isUnicodeIdentifierStart(char)`, `isUpperCase(char)`

isLetter

```
public static boolean isLetter(int codePoint)
```

Determines if the specified character (Unicode code point) is a letter.

A character is considered to be a letter if its general category type, provided by `getType(codePoint)`, is any of the following:

- `UPPERCASE_LETTER`
- `LOWERCASE_LETTER`
- `TITLECASE_LETTER`
- `MODIFIER_LETTER`
- `OTHER_LETTER`

Not all letters have case. Many characters are letters but are neither uppercase nor lowercase nor titlecase.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is a letter; `false` otherwise.

Since:

1.5

See Also:

`isDigit(int)`, `isJavaIdentifierStart(int)`, `isLetterOrDigit(int)`, `isLowerCase(int)`, `isTitleCase(int)`, `isUnicodeIdentifierStart(int)`, `isUpperCase(int)`

isLetterOrDigit

```
public static boolean isLetterOrDigit(char ch)
```

Determines if the specified character is a letter or digit.

A character is considered to be a letter or digit if either `Character.isLetter(char ch)` or `Character.isDigit(char ch)` returns `true` for the character.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isLetterOrDigit(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character is a letter or digit; `false` otherwise.

Since:

1.0.2

See Also:

`isDigit(char)`, `isJavaIdentifierPart(char)`, `isJavaLetter(char)`, `isJavaLetterOrDigit(char)`, `isLetter(char)`, `isUnicodeIdentifierPart(char)`

isLetterOrDigit

```
public static boolean isLetterOrDigit(int codePoint)
```

Determines if the specified character (Unicode code point) is a letter or digit.

A character is considered to be a letter or digit if either `isLetter(codePoint)` or `isDigit(codePoint)` returns `true` for the character.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is a letter or digit; `false` otherwise.

Since:

1.5

See Also:

`isDigit(int)`, `isJavaIdentifierPart(int)`, `isLetter(int)`, `isUnicodeIdentifierPart(int)`

isJavaLetter

```
@Deprecated(since="1.1")
public static boolean isJavaLetter(char ch)
```

Deprecated.
Replaced by isJavaIdentifierStart(char).

Determines if the specified character is permissible as the first character in a Java identifier.

A character may start a Java identifier if and only if one of the following conditions is true:

- `isLetter(ch)` returns `true`
- `getType(ch)` returns `LETTER_NUMBER`
- `ch` is a currency symbol (such as `'$'`)
- `ch` is a connecting punctuation character (such as `'_'`).

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character may start a Java identifier; `false` otherwise.

Since:

1.0.2

See Also:

`isJavaLetterOrDigit(char)`, `isJavaIdentifierStart(char)`, `isJavaIdentifierPart(char)`, `isLetter(char)`, `isLetterOrDigit(char)`, `isUnicodeIdentifierStart(char)`

isJavaLetterOrDigit

```
@Deprecated(since="1.1")
public static boolean isJavaLetterOrDigit(char ch)
```

Deprecated.
Replaced by isJavaIdentifierPart(char).

Determines if the specified character may be part of a Java identifier as other than the first character.

A character may be part of a Java identifier if and only if one of the following conditions is true:

- it is a letter
- it is a currency symbol (such as `'$'`)
- it is a connecting punctuation character (such as `'_'`)
- it is a digit
- it is a numeric letter (such as a Roman numeral character)
- it is a combining mark
- it is a non-spacing mark
- `isIdentifierIgnorable` returns `true` for the character.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character may be part of a Java identifier; `false` otherwise.

Since:

1.0.2

See Also:

`isJavaLetter(char)`, `isJavaIdentifierStart(char)`, `isJavaIdentifierPart(char)`, `isLetter(char)`, `isLetterOrDigit(char)`, `isUnicodeIdentifierPart(char)`, `isIdentifierIgnorable(char)`

isAlphabetic

```
public static boolean isAlphabetic(int codePoint)
```

Determines if the specified character (Unicode code point) is alphabetic.

A character is considered to be alphabetic if its general category type, provided by `getType(codePoint)`, is any of the following:

- `UPPERCASE_LETTER`
- `LOWERCASE_LETTER`
- `TITLECASE_LETTER`
- `MODIFIER_LETTER`
- `OTHER_LETTER`
- `LETTER_NUMBER`

or it has contributory property Other_Alphabetic as defined by the Unicode Standard.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is a Unicode alphabet character, `false` otherwise.

Since:

1.7

isIdeographic

```
public static boolean isIdeographic(int codePoint)
```

Determines if the specified character (Unicode code point) is a CJKV (Chinese, Japanese, Korean and Vietnamese) ideograph, as defined by the Unicode Standard.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is a Unicode ideograph character, `false` otherwise.

Since:

1.7

isJavaIdentifierStart

```
public static boolean isJavaIdentifierStart(char ch)
```

Determines if the specified character is permissible as the first character in a Java identifier.

A character may start a Java identifier if and only if one of the following conditions is true:

- `isLetter(ch)` returns `true`
- `getType(ch)` returns `LETTER_NUMBER`
- `ch` is a currency symbol (such as `'$'`)
- `ch` is a connecting punctuation character (such as `'_'`).

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isJavaIdentifierStart(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character may start a Java identifier; `false` otherwise.

Since:

1.1

See Also:

`isJavaIdentifierPart(char)`,
`isLetter(char)`,
`isUnicodeIdentifierStart(char)`,
`SourceVersion.isIdentifier(CharSequence)`

isJavaIdentifierStart

```
public static boolean isJavaIdentifierStart(int codePoint)
```

Determines if the character (Unicode code point) is permissible as the first character in a Java identifier.

A character may start a Java identifier if and only if one of the following conditions is true:

- `isLetter(codePoint)` returns `true`
- `getType(codePoint)` returns `LETTER_NUMBER`
- the referenced character is a currency symbol (such as `'$'`)
- the referenced character is a connecting punctuation character (such as `'_'`).

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character may start a Java identifier; `false` otherwise.

Since:

1.5

See Also:

`isJavaIdentifierPart(int)`,
`isLetter(int)`,
`isUnicodeIdentifierStart(int)`,
`SourceVersion.isIdentifier(CharSequence)`

isJavaIdentifierPart

```
public static boolean isJavaIdentifierPart(char ch)
```

Determines if the specified character may be part of a Java identifier as other than the first character.

A character may be part of a Java identifier if any of the following conditions are true:

- it is a letter
- it is a currency symbol (such as `'$'`)
- it is a connecting punctuation character (such as `'_'`)
- it is a digit
- it is a numeric letter (such as a Roman numeral character)
- it is a combining mark
- it is a non-spacing mark
- `isIdentifierIgnorable` returns `true` for the character

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isJavaIdentifierPart(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character may be part of a Java identifier; `false` otherwise.

Since:

1.1

See Also:

`isIdentifierIgnorable(char)`,
`isJavaIdentifierStart(char)`,
`isLetterOrDigit(char)`,
`isUnicodeIdentifierPart(char)`,
`SourceVersion.isIdentifier(CharSequence)`

isJavaIdentifierPart

```
public static boolean isJavaIdentifierPart(int codePoint)
```

Determines if the character (Unicode code point) may be part of a Java identifier as other than the first character.

A character may be part of a Java identifier if any of the following conditions are true:

- it is a letter
- it is a currency symbol (such as `'$'`)
- it is a connecting punctuation character (such as `'_'`)
- it is a digit
- it is a numeric letter (such as a Roman numeral character)
- it is a combining mark

- it is a non-spacing mark
- `isIdentifierIgnorable(codePoint)` returns `true` for the code point

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character may be part of a Java identifier; `false` otherwise.

Since:

1.5

See Also:

`isIdentifierIgnorable(int)`,
`isJavaIdentifierStart(int)`,
`isLetterOrDigit(int)`,
`isUnicodeIdentifierPart(int)`,
`SourceVersion.isIdentifier(CharSequence)`

isUnicodeIdentifierStart

```
public static boolean isUnicodeIdentifierStart(char ch)
```

Determines if the specified character is permissible as the first character in a Unicode identifier.

A character may start a Unicode identifier if and only if one of the following conditions is true:

- `isLetter(ch)` returns `true`
- `getType(ch)` returns `LETTER_NUMBER`.
- it is an `Other_ID_Start` character.

This method conforms to UAX31-R1: Default Identifiers requirement of the Unicode Standard, with the following profile of UAX31:

```
Start := ID_Start + 'VERTICAL TILDE' (U+2E2F)
```

`'VERTICAL TILDE'` is added to `Start` for backward compatibility.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isUnicodeIdentifierStart(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character may start a Unicode identifier; `false` otherwise.

Since:

1.1

External Specifications

Unicode Character Database,
Unicode Identifier and Pattern Syntax

See Also:

`isJavaIdentifierStart(char)`, `isLetter(char)`,
`isUnicodeIdentifierPart(char)`

isUnicodeIdentifierStart

```
public static boolean isUnicodeIdentifierStart(int codePoint)
```

Determines if the specified character (Unicode code point) is permissible as the first character in a Unicode identifier.

A character may start a Unicode identifier if and only if one of the following conditions is true:

- `isLetter(codePoint)` returns `true`
- `getType(codePoint)` returns `LETTER_NUMBER`.
- it is an `Other_ID_Start` character.

This method conforms to UAX31-R1: Default Identifiers requirement of the Unicode Standard, with the following profile of UAX31:

```
Start := ID_Start + 'VERTICAL TILDE' (U+2E2F)
```

`'VERTICAL TILDE'` is added to `Start` for backward compatibility.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character may start a Unicode identifier; `false` otherwise.

Since:

1.5

External Specifications

Unicode Character Database,
Unicode Identifier and Pattern Syntax

See Also:

`isJavaIdentifierStart(int)`, `isLetter(int)`,
`isUnicodeIdentifierPart(int)`

isUnicodeIdentifierPart

`public static boolean isUnicodeIdentifierPart(char ch)`

Determines if the specified character may be part of a Unicode identifier as other than the first character.

A character may be part of a Unicode identifier if and only if one of the following statements is true:

- it is a letter
- it is a connecting punctuation character (such as `'_'`)
- it is a digit
- it is a numeric letter (such as a Roman numeral character)
- it is a combining mark
- it is a non-spacing mark
- `isIdentifierIgnorable` returns `true` for this character.
- it is an `Other_ID_Start` character.
- it is an `Other_ID_Continue` character.

This method conforms to UAX31-R1: Default Identifiers requirement of the Unicode Standard, with the following profile of UAX31:

```
Continue := Start + ID_Continue + ignorable
Medial := empty
ignorable := isIdentifierIgnorable(char) returns true for the character
```

`ignorable` is added to `Continue` for backward compatibility.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isUnicodeIdentifierPart(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character may be part of a Unicode identifier; `false` otherwise.

Since:

1.1

External Specifications

Unicode Character Database,
Unicode Identifier and Pattern Syntax

See Also:

`isIdentifierIgnorable(char)`, `isJavaIdentifierPart(char)`, `isLetterOrDigit(char)`, `isUnicodeIdentifierStart(char)`

isUnicodeIdentifierPart

```
public static boolean isUnicodeIdentifierPart(int codePoint)
```

Determines if the specified character (Unicode code point) may be part of a Unicode identifier as other than the first character.

A character may be part of a Unicode identifier if and only if one of the following statements is true:

- it is a letter
- it is a connecting punctuation character (such as `'_'`)
- it is a digit
- it is a numeric letter (such as a Roman numeral character)
- it is a combining mark
- it is a non-spacing mark
- `isIdentifierIgnorable` returns `true` for this character.
- it is an `Other_ID_Start` character.
- it is an `Other_ID_Continue` character.

This method conforms to UAX31-R1: Default Identifiers requirement of the Unicode Standard, with the following profile of UAX31:

```
Continue := Start + ID_Continue + ignorable
Medial := empty
ignorable := isIdentifierIgnorable(int) returns true for the character
```

`ignorable` is added to `Continue` for backward compatibility.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character may be part of a Unicode identifier; `false` otherwise.

Since:

1.5

External Specifications

Unicode Character Database,
Unicode Identifier and Pattern Syntax

See Also:

`isIdentifierIgnorable(int)`, `isJavaIdentifierPart(int)`, `isLetterOrDigit(int)`, `isUnicodeIdentifierStart(int)`

isIdentifierIgnorable

```
public static boolean isIdentifierIgnorable(char ch)
```

Determines if the specified character should be regarded as an ignorable character in a Java identifier or a Unicode identifier.

The following Unicode characters are ignorable in a Java identifier or a Unicode identifier:

- ISO control characters that are not whitespace
  - `'\u0000'` through `'\u0008'`
  - `'\u000E'` through `'\u001B'`
  - `'\u007F'` through `'\u009F'`
- all characters that have the `FORMAT` general category value

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isIdentifierIgnorable(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character is an ignorable control character that may be part of a Java or Unicode identifier; `false` otherwise.

Since:

1.1

See Also:

`isJavaIdentifierPart(char)`, `isUnicodeIdentifierPart(char)`

isIdentifierIgnorable

```
public static boolean isIdentifierIgnorable(int codePoint)
```

Determines if the specified character (Unicode code point) should be regarded
as an ignorable character in a Java identifier or a Unicode identifier.

The following Unicode characters are ignorable in a Java identifier or a Unicode identifier:

- ISO control characters that are not whitespace
  - `'\u0000'` through `'\u0008'`
  - `'\u000E'` through `'\u001B'`
  - `'\u007F'` through `'\u009F'`
- all characters that have the `FORMAT` general category value

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is an ignorable control character that may be part of a Java or Unicode identifier; `false` otherwise.

Since:

1.5

See Also:

`isJavaIdentifierPart(int)`, `isUnicodeIdentifierPart(int)`

isEmoji

```
public static boolean isEmoji(int codePoint)
```

Determines if the specified character (Unicode code point) is an Emoji.

A character is considered to be an Emoji if and only if it has the `Emoji` property, defined in Unicode Emoji (Technical Standard #51).

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is an Emoji; `false` otherwise.

Since:

21

External Specifications

Unicode Emoji

isEmojiPresentation

```
public static boolean isEmojiPresentation(int codePoint)
```

Determines if the specified character (Unicode code point) has the Emoji Presentation property by default.

A character is considered to have the Emoji Presentation property if and only if it has the `Emoji_Presentation` property, defined in Unicode Emoji (Technical Standard #51).

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character has the Emoji Presentation property; `false` otherwise.

Since:

21

External Specifications

Unicode Emoji

isEmojiModifier

```
public static boolean isEmojiModifier(int codePoint)
```

Determines if the specified character (Unicode code point) is an Emoji Modifier.

A character is considered to be an Emoji Modifier if and only if it has the `Emoji_Modifier` property, defined in Unicode Emoji (Technical Standard #51).

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is an Emoji Modifier; `false` otherwise.

Since:

21

External Specifications

Unicode Emoji

isEmojiModifierBase

```
public static boolean isEmojiModifierBase(int codePoint)
```

Determines if the specified character (Unicode code point) is an Emoji Modifier Base.

A
character is considered to be an Emoji Modifier Base if and only if it has the `Emoji_Modifier_Base` property, defined in Unicode Emoji (Technical Standard #51).

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is an Emoji Modifier Base; `false` otherwise.

Since:

21

External Specifications

Unicode Emoji

isEmojiComponent

```
public static boolean isEmojiComponent(int codePoint)
```

Determines if the specified character (Unicode code point) is an Emoji Component.

A character is considered to be an Emoji Component if and only if it has the `Emoji_Component` property, defined in Unicode Emoji (Technical Standard #51).

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is an Emoji Component; `false` otherwise.

Since:

21

External Specifications

Unicode Emoji

isExtendedPictographic

```
public static boolean isExtendedPictographic(int codePoint)
```

Determines if the specified character (Unicode code point) is an Extended Pictographic.

A character is considered to be an Extended Pictographic if and only if it has the `Extended_Pictographic` property, defined in Unicode Emoji (Technical Standard #51).

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is an Extended Pictographic; `false` otherwise.

Since:

21

External Specifications

Unicode Emoji

toLowerCase

```
public static char toLowerCase(char ch)
```

Converts the character argument to lowercase using case mapping information from the UnicodeData file.

Note that `Character.isLowerCase(Character.toLowerCase(ch))` does not always return `true` for some ranges of characters, particularly those that are symbols or ideographs.

In general, `String.toLowerCase()` should be used to map characters to lowercase. `String` case mapping methods have several benefits over `Character` case mapping methods. `String` case mapping methods can perform locale-sensitive mappings, context-sensitive mappings, and 1:M character mappings, whereas the `Character` case mapping methods cannot.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `toLowerCase(int)` method.

Parameters:

`ch` - the character to be converted.

Returns:

the lowercase equivalent of the character, if any; otherwise, the character itself.

See Also:

`isLowerCase(char)`, `String.toLowerCase()`

toLowerCase

```
public static int toLowerCase(int codePoint)
```

Converts the character (Unicode code point) argument to lowercase using case mapping information from the UnicodeData file.

Note that `Character.isLowerCase(Character.toLowerCase(codePoint))` does not always return `true` for some ranges of characters, particularly those that are symbols or ideographs.

In general, `String.toLowerCase()` should be used to map characters to lowercase. `String` case mapping methods have several benefits over `Character` case mapping methods. `String` case mapping methods can perform locale-sensitive mappings, context-sensitive mappings, and 1:M character mappings, whereas the `Character` case mapping methods cannot.

Parameters:

`codePoint` - the character (Unicode code point) to be converted.

Returns:

the lowercase equivalent of the character (Unicode code point), if any; otherwise, the character itself.

Since:

1.5

See Also:

`isLowerCase(int)`, `String.toLowerCase()`

toUpperCase

```
public static char toUpperCase(char ch)
```

Converts the character argument to uppercase using case mapping information from the UnicodeData file.

Note that `Character.isUpperCase(Character.toUpperCase(ch))` does not always return `true` for some ranges of characters, particularly those that are symbols or ideographs.

In general, `String.toUpperCase()` should be used to map characters to uppercase. `String` case mapping methods have several benefits over `Character` case mapping methods. `String` case mapping methods can perform locale-sensitive mappings, context-sensitive mappings, and 1:M character mappings, whereas the `Character` case mapping methods cannot.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `toUpperCase(int)` method.

Parameters:

`ch` - the character to be converted.

Returns:

the uppercase equivalent of the character, if any; otherwise, the character itself.

See Also:

`isUpperCase(char)`, `String.toUpperCase()`

toUpperCase

```
public static int toUpperCase(int codePoint)
```

Converts the character (Unicode code point) argument to uppercase using case mapping information from the UnicodeData file.

Note that `Character.isUpperCase(Character.toUpperCase(codePoint))` does not always return `true` for some ranges of characters, particularly those that are symbols or ideographs.

In general, `String.toUpperCase()` should be used to map characters to uppercase. `String` case mapping methods have several benefits over `Character` case mapping methods. `String` case mapping methods can perform locale-sensitive mappings, context-sensitive mappings, and 1:M character mappings, whereas the `Character` case mapping methods cannot.

Parameters:

`codePoint` - the character (Unicode code point) to be converted.

Returns:

the uppercase equivalent of the character, if any; otherwise, the character itself.

Since:

1.5

See Also:

`isUpperCase(int)`, `String.toUpperCase()`

toTitleCase

```
public static char toTitleCase(char ch)
```

Converts the character argument to titlecase using case mapping information from the UnicodeData file. If a character has no explicit titlecase mapping and is not itself a titlecase char according to UnicodeData, then the uppercase mapping is returned as an equivalent titlecase mapping. If the `char` argument is already a titlecase `char`, the same `char` value will be returned.

Note that `Character.isTitleCase(Character.toTitleCase(ch))` does not always return `true` for some ranges of characters.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `toTitleCase(int)` method.

Parameters:

`ch` - the character to be converted.

Returns:

the titlecase equivalent of the character, if any; otherwise, the character itself.

Since:

1.0.2

See Also:

`isTitleCase(char)`, `toLowerCase(char)`, `toUpperCase(char)`

toTitleCase

```
public static int toTitleCase(int codePoint)
```

Converts the character (Unicode code point) argument to titlecase using case mapping information from the UnicodeData file. If a character has no explicit titlecase mapping and is not itself a titlecase char according to UnicodeData, then the uppercase mapping is returned as an equivalent titlecase mapping. If the character argument is already a titlecase character, the same character value will be returned.

Note that `Character.isTitleCase(Character.toTitleCase(codePoint))` does not always return `true` for some ranges of characters.

Parameters:

`codePoint` - the character (Unicode code point) to be converted.

Returns:

the titlecase equivalent of the character, if any; otherwise, the character itself.

Since:

1.5

See Also:

`isTitleCase(int)`, `toLowerCase(int)`, `toUpperCase(int)`

digit

```
public static int digit(char ch,
                        int radix)
```

Returns the numeric value of the character `ch` in the specified radix.

If the radix is not in the range `MIN_RADIX` ≤ `radix` ≤ `MAX_RADIX` or if the value of `ch` is not a valid digit in the specified radix, `-1` is returned. A character is a valid digit if at least one of the following is true:

- The method `isDigit` is `true` of the character and the Unicode decimal digit value of the character (or its single-character decomposition) is less than the specified radix. In this case the decimal digit value is returned.
- The character is one of the uppercase Latin letters `'A'` through `'Z'` and its code is less than `radix + 'A' - 10`. In this case, `ch - 'A' + 10` is returned.
- The character is one of the lowercase Latin letters `'a'` through `'z'` and its code is less than `radix + 'a' - 10`. In this case, `ch - 'a' + 10` is returned.
- The character is one of the fullwidth uppercase Latin letters A (`'\uFF21'`) through Z (`'\uFF3A'`) and its code is less than `radix + '\uFF21' - 10`. In this case, `ch - '\uFF21' + 10` is returned.
- The character is one of the fullwidth lowercase Latin letters a (`'\uFF41'`) through z (`'\uFF5A'`) and its code is less than `radix + '\uFF41' - 10`. In this case, `ch - '\uFF41' + 10` is returned.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `digit(int, int)` method.

Parameters:

`ch` - the character to be converted.

`radix` - the radix.

Returns:

the numeric value represented by the character in the specified radix.

See Also:

`forDigit(int, int)`,
`isDigit(char)`

digit

```
public static int digit(int codePoint,
                        int radix)
```

Returns the numeric value of the specified character (Unicode code point) in the specified radix.

If the radix is not in the range `MIN_RADIX` ≤ `radix` ≤ `MAX_RADIX` or if the character is not a valid digit in the specified radix, `-1` is returned. A character is a valid digit if at least one of the following is true:

- The method `isDigit(codePoint)` is `true` of the character and the Unicode decimal digit value of the character (or its single-character decomposition) is less than the specified radix. In this case the decimal digit value is returned.
- The character is one of the uppercase Latin letters `'A'` through `'Z'` and its code is less than `radix + 'A' - 10`. In this case, `codePoint - 'A' + 10` is returned.
- The character is one of the lowercase Latin letters `'a'` through `'z'` and its code is less than `radix + 'a' - 10`. In this case, `codePoint - 'a' + 10` is returned.
- The character is one of the fullwidth uppercase Latin letters A (`'\uFF21'`) through Z (`'\uFF3A'`) and its code is less than `radix + '\uFF21' - 10`. In this case, `codePoint - '\uFF21' + 10` is returned.
- The character is one of the fullwidth lowercase Latin letters a (`'\uFF41'`) through z (`'\uFF5A'`) and its code is less than `radix + '\uFF41'- 10`. In this case, `codePoint - '\uFF41' + 10` is returned.

Parameters:

`codePoint` - the character (Unicode code point) to be converted.

`radix` - the radix.

Returns:

the numeric value represented by the character in the specified radix.

Since:

1.5

See Also:

`forDigit(int, int)`,
`isDigit(int)`

getNumericValue

```
public static int getNumericValue(char ch)
```

Returns the `int` value that the specified Unicode character represents. For example, the character `'\u216C'` (the roman numeral fifty) will return an int with a value of 50.

The letters A-Z in their uppercase (`'\u0041'` through `'\u005A'`), lowercase (`'\u0061'` through `'\u007A'`), and full width variant (`'\uFF21'` through `'\uFF3A'` and `'\uFF41'` through `'\uFF5A'`) forms have numeric values from 10

through 35. This is independent of the Unicode specification, which does not assign numeric values to these `char` values.

If the character does not have a numeric value, then -1 is returned. If the character has a numeric value that cannot be represented as a nonnegative integer (for example, a fractional value), then -2 is returned.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `getNumericValue(int)` method.

Parameters:

`ch` - the character to be converted.

Returns:

the numeric value of the character, as a nonnegative `int` value; -2 if the character has a numeric value but the value can not be represented as a nonnegative `int` value; -1 if the character has no numeric value.

Since:

1.1

See Also:

`forDigit(int, int)`,
`isDigit(char)`

getNumericValue

```
public static int getNumericValue(int codePoint)
```

Returns the `int` value that the specified character (Unicode code point) represents. For example, the character `'\u216C'` (the Roman numeral fifty) will return an `int` with a value of 50.

The letters A-Z in their uppercase (`'\u0041'` through `'\u005A'`), lowercase (`'\u0061'` through `'\u007A'`), and full width variant (`'\uFF21'` through `'\uFF3A'` and `'\uFF41'` through `'\uFF5A'`) forms have numeric values from 10 through 35. This is independent of the Unicode specification, which does not assign numeric values to these `char` values.

If the character does not have a numeric value, then -1 is returned. If the character has a numeric value that cannot be represented as a nonnegative integer (for example, a fractional value), then -2 is returned.

Parameters:

`codePoint` - the character (Unicode code point) to be converted.

Returns:

the numeric value of the character, as a nonnegative `int` value; -2 if the character has a numeric value but the value can not be represented as a nonnegative `int` value; -1 if the character has no numeric value.

Since:

1.5

See Also:

`forDigit(int, int)`,
`isDigit(int)`

isSpace

```
@Deprecated(since="1.1")
public static boolean isSpace(char ch)
```

Deprecated.
Replaced by isWhitespace(char).

Determines if the specified character is ISO-LATIN-1 white space. This method returns `true` for the following five characters only:

Character	Code	Name
`'\t'`	U+0009	HORIZONTAL TABULATION
`'\n'`	U+000A	NEW LINE
`'\f'`	U+000C	FORM FEED
`'\r'`	U+000D	CARRIAGE RETURN
`' '`	U+0020	SPACE

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character is ISO-LATIN-1 white space; `false` otherwise.

See Also:

`isSpaceChar(char)`, `isWhitespace(char)`

isSpaceChar

```
public static boolean isSpaceChar(char ch)
```

Determines if the specified character is a Unicode space character. A character is considered to be a space character if and only if it is specified to be a space character by the Unicode Standard. This method returns true if the character's general category type is any of the following:

- `SPACE_SEPARATOR`

- LINE_SEPARATOR
- PARAGRAPH_SEPARATOR

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isSpaceChar(int)` method.

Parameters:

ch - the character to be tested.

Returns:

`true` if the character is a space character; `false` otherwise.

Since:

1.1

See Also:

`isWhitespace(char)`

isSpaceChar

```
public static boolean isSpaceChar(int codePoint)
```

Determines if the specified character (Unicode code point) is a Unicode space character. A character is considered to be a space character if and only if it is specified to be a space character by the Unicode Standard. This method returns true if the character's general category type is any of the following:

- SPACE_SEPARATOR
- LINE_SEPARATOR
- PARAGRAPH_SEPARATOR

Parameters:

codePoint - the character (Unicode code point) to be tested.

Returns:

`true` if the character is a space character; `false` otherwise.

Since:

1.5

See Also:

`isWhitespace(int)`

isWhitespace

```
public static boolean isWhitespace(char ch)
```

Determines if the specified character is white space according to Java. A character is a Java whitespace character if and only if it satisfies one of the following criteria:

- It is a Unicode space character (`SPACE_SEPARATOR`, `LINE_SEPARATOR`, or `PARAGRAPH_SEPARATOR`) but is not also a non-breaking space (`'\u00A0'`, `'\u2007'`, `'\u202F'`).
- It is `'\t'`, U+0009 HORIZONTAL TABULATION.
- It is `'\n'`, U+000A LINE FEED.
- It is `'\u000B'`, U+000B VERTICAL TABULATION.
- It is `'\f'`, U+000C FORM FEED.
- It is `'\r'`, U+000D CARRIAGE RETURN.
- It is `'\u001C'`, U+001C FILE SEPARATOR.
- It is `'\u001D'`, U+001D GROUP SEPARATOR.
- It is `'\u001E'`, U+001E RECORD SEPARATOR.
- It is `'\u001F'`, U+001F UNIT SEPARATOR.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isWhitespace(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character is a Java whitespace character; `false` otherwise.

Since:

1.1

See Also:

`isSpaceChar(char)`

isWhitespace

```
public static boolean isWhitespace(int codePoint)
```

Determines if the specified character (Unicode code point) is white space according to Java. A character is a Java whitespace character if and only if it satisfies one of the following criteria:

- It is a Unicode space character (`SPACE_SEPARATOR`, `LINE_SEPARATOR`, or `PARAGRAPH_SEPARATOR`) but is not also a non-breaking space (`'\u00A0'`, `'\u2007'`, `'\u202F'`).
- It is `'\t'`, U+0009 HORIZONTAL TABULATION.
- It is `'\n'`, U+000A LINE FEED.
- It is `'\u000B'`, U+000B VERTICAL TABULATION.
- It is `'\f'`, U+000C FORM FEED.

- It is `'\r'`, U+000D CARRIAGE RETURN.
- It is `'\u001C'`, U+001C FILE SEPARATOR.
- It is `'\u001D'`, U+001D GROUP SEPARATOR.
- It is `'\u001E'`, U+001E RECORD SEPARATOR.
- It is `'\u001F'`, U+001F UNIT SEPARATOR.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is a Java whitespace character; `false` otherwise.

Since:

1.5

See Also:

`isSpaceChar(int)`

isISOControl

```
public static boolean isISOControl(char ch)
```

Determines if the specified character is an ISO control character. A character is considered to be an ISO control character if its code is in the range `'\u0000'` through `'\u001F'` or in the range `'\u007F'` through `'\u009F'`.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isISOControl(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

`true` if the character is an ISO control character; `false` otherwise.

Since:

1.1

See Also:

`isSpaceChar(char)`, `isWhitespace(char)`

isISOControl

```
public static boolean isISOControl(int codePoint)
```

Determines if the referenced character (Unicode code point) is an ISO control character. A character is considered to be an ISO control character if its code is in the range `'\u0000'` through `'\u001F'` or in the range `'\u007F'` through `'\u009F'`.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is an ISO control character; `false` otherwise.

Since:

1.5

See Also:

`isSpaceChar(int)`, `isWhitespace(int)`

getType

```
public static int getType(char ch)
```

Returns a value indicating a character's general category.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `getType(int)` method.

Parameters:

`ch` - the character to be tested.

Returns:

a value of type `int` representing the character's general category.

Since:

1.1

See Also:

`COMBINING_SPACING_MARK`, `CONNECTOR_PUNCTUATION`, `CONTROL`, `CURRENCY_SYMBOL`, `DASH_PUNCTUATION`, `DECIMAL_DIGIT_NUMBER`, `ENCLOSING_MARK`, `END_PUNCTUATION`, `FINAL_QUOTE_PUNCTUATION`, `FORMAT`, `INITIAL_QUOTE_PUNCTUATION`, `LETTER_NUMBER`, `LINE_SEPARATOR`, `LOWERCASE_LETTER`, `MATH_SYMBOL`, `MODIFIER_LETTER`, `MODIFIER_SYMBOL`, `NON_SPACING_MARK`, `OTHER_LETTER`, `OTHER_NUMBER`, `OTHER_PUNCTUATION`, `OTHER_SYMBOL`, `PARAGRAPH_SEPARATOR`, `PRIVATE_USE`, `SPACE_SEPARATOR`, `START_PUNCTUATION`, `SURROGATE`, `TITLECASE_LETTER`, `UNASSIGNED`, `UPPERCASE_LETTER`

getType

```
public static int getType(int codePoint)
```

Returns a value indicating a character's general category.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

a value of type `int` representing the character's general category.

Since:

1.5

See Also:

`COMBINING_SPACING_MARK`, `CONNECTOR_PUNCTUATION`, `CONTROL`, `CURRENCY_SYMBOL`, `DASH_PUNCTUATION`, `DECIMAL_DIGIT_NUMBER`, `ENCLOSING_MARK`, `END_PUNCTUATION`, `FINAL_QUOTE_PUNCTUATION`, `FORMAT`, `INITIAL_QUOTE_PUNCTUATION`, `LETTER_NUMBER`, `LINE_SEPARATOR`, `LOWERCASE_LETTER`, `MATH_SYMBOL`, `MODIFIER_LETTER`, `MODIFIER_SYMBOL`, `NON_SPACING_MARK`, `OTHER_LETTER`, `OTHER_NUMBER`, `OTHER_PUNCTUATION`, `OTHER_SYMBOL`, `PARAGRAPH_SEPARATOR`, `PRIVATE_USE`, `SPACE_SEPARATOR`, `START_PUNCTUATION`, `SURROGATE`, `TITLECASE_LETTER`, `UNASSIGNED`, `UPPERCASE_LETTER`

forDigit

```
public static char forDigit(int digit,
                            int radix)
```

Determines the character representation for a specific digit in the specified radix. If the value of `radix` is not a valid radix, or the value of `digit` is not a valid digit in the specified radix, the null character (`'\u0000'`) is returned.

The `radix` argument is valid if it is greater than or equal to `MIN_RADIX` and less than or equal to `MAX_RADIX`. The `digit` argument is valid if `0 <= digit < radix`.

If the digit is less than 10, then `'0' + digit` is returned. Otherwise, the value `'a' + digit - 10` is returned.

Parameters:

`digit` - the number to convert to a character.

`radix` - the radix.

Returns:

the `char` representation of the specified digit in the specified radix.

See Also:

`MIN_RADIX`,
`MAX_RADIX`,
`digit(char, int)`

getDirectionality

```
public static byte getDirectionality(char ch)
```

Returns the Unicode directionality property for the given character. Character directionality is used to calculate the visual ordering of text. The directionality value of undefined `char` values is `DIRECTIONALITY_UNDEFINED`.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `getDirectionality(int)` method.

Parameters:

`ch` - `char` for which the directionality property is requested.

Returns:

the directionality property of the `char` value.

Since:

1.4

See Also:

`DIRECTIONALITY_UNDEFINED`,
`DIRECTIONALITY_LEFT_TO_RIGHT`,
`DIRECTIONALITY_RIGHT_TO_LEFT`,
`DIRECTIONALITY_RIGHT_TO_LEFT_ARABIC`,
`DIRECTIONALITY_EUROPEAN_NUMBER`,
`DIRECTIONALITY_EUROPEAN_NUMBER_SEPARATOR`,
`DIRECTIONALITY_EUROPEAN_NUMBER_TERMINATOR`,
`DIRECTIONALITY_ARABIC_NUMBER`,
`DIRECTIONALITY_COMMON_NUMBER_SEPARATOR`,
`DIRECTIONALITY_NONSPACING_MARK`,
`DIRECTIONALITY_BOUNDARY_NEUTRAL`,
`DIRECTIONALITY_PARAGRAPH_SEPARATOR`,
`DIRECTIONALITY_SEGMENT_SEPARATOR`,
`DIRECTIONALITY_WHITESPACE`,
`DIRECTIONALITY_OTHER_NEUTRALS`,
`DIRECTIONALITY_LEFT_TO_RIGHT_EMBEDDING`,
`DIRECTIONALITY_LEFT_TO_RIGHT_OVERRIDE`,
`DIRECTIONALITY_RIGHT_TO_LEFT_EMBEDDING`,
`DIRECTIONALITY_RIGHT_TO_LEFT_OVERRIDE`,
`DIRECTIONALITY_POP_DIRECTIONAL_FORMAT`,
`DIRECTIONALITY_LEFT_TO_RIGHT_ISOLATE`,
`DIRECTIONALITY_RIGHT_TO_LEFT_ISOLATE`,
`DIRECTIONALITY_FIRST_STRONG_ISOLATE`,
`DIRECTIONALITY_POP_DIRECTIONAL_ISOLATE`

getDirectionality

```
public static byte getDirectionality(int codePoint)
```

Returns the Unicode directionality property for the given character (Unicode code point). Character directionality is used to calculate the visual ordering of text. The directionality value of undefined character is `DIRECTIONALITY_UNDEFINED`.

Parameters:

`codePoint` - the character (Unicode code point) for which the directionality property is requested.

Returns:

the directionality property of the character.

Since:

1.5

See Also:

`DIRECTIONALITY_UNDEFINED`,
`DIRECTIONALITY_LEFT_TO_RIGHT`,
`DIRECTIONALITY_RIGHT_TO_LEFT`,
`DIRECTIONALITY_RIGHT_TO_LEFT_ARABIC`,
`DIRECTIONALITY_EUROPEAN_NUMBER`,
`DIRECTIONALITY_EUROPEAN_NUMBER_SEPARATOR`,
`DIRECTIONALITY_EUROPEAN_NUMBER_TERMINATOR`,
`DIRECTIONALITY_ARABIC_NUMBER`,
`DIRECTIONALITY_COMMON_NUMBER_SEPARATOR`,
`DIRECTIONALITY_NONSPACING_MARK`,
`DIRECTIONALITY_BOUNDARY_NEUTRAL`,
`DIRECTIONALITY_PARAGRAPH_SEPARATOR`,
`DIRECTIONALITY_SEGMENT_SEPARATOR`,
`DIRECTIONALITY_WHITESPACE`,
`DIRECTIONALITY_OTHER_NEUTRALS`,
`DIRECTIONALITY_LEFT_TO_RIGHT_EMBEDDING`,
`DIRECTIONALITY_LEFT_TO_RIGHT_OVERRIDE`,
`DIRECTIONALITY_RIGHT_TO_LEFT_EMBEDDING`,
`DIRECTIONALITY_RIGHT_TO_LEFT_OVERRIDE`,
`DIRECTIONALITY_POP_DIRECTIONAL_FORMAT`,
`DIRECTIONALITY_LEFT_TO_RIGHT_ISOLATE`,
`DIRECTIONALITY_RIGHT_TO_LEFT_ISOLATE`,
`DIRECTIONALITY_FIRST_STRONG_ISOLATE`,
`DIRECTIONALITY_POP_DIRECTIONAL_ISOLATE`

isMirrored

```
public static boolean isMirrored(char ch)
```

Determines whether the character is mirrored according to the Unicode specification. Mirrored characters should have their glyphs horizontally

mirrored when displayed in text that is right-to-left. For example, `'\u0028'` LEFT PARENTHESIS is semantically defined to be an *opening parenthesis*. This will appear as a "(" in text that is left-to-right but as a ")" in text that is right-to-left.

Note: This method cannot handle supplementary characters. To support all Unicode characters, including supplementary characters, use the `isMirrored(int)` method.

Parameters:

`ch` - `char` for which the mirrored property is requested

Returns:

`true` if the char is mirrored, `false` if the `char` is not mirrored or is not defined.

Since:

1.4

isMirrored

```
public static boolean isMirrored(int codePoint)
```

Determines whether the specified character (Unicode code point) is mirrored according to the Unicode specification. Mirrored characters should have their glyphs horizontally mirrored when displayed in text that is right-to-left. For example, `'\u0028'` LEFT PARENTHESIS is semantically defined to be an *opening parenthesis*. This will appear as a "(" in text that is left-to-right but as a ")" in text that is right-to-left.

Parameters:

`codePoint` - the character (Unicode code point) to be tested.

Returns:

`true` if the character is mirrored, `false` if the character is not mirrored or is not defined.

Since:

1.5

compareTo

```
public int compareTo(Character anotherCharacter)
```

Compares two `Character` objects numerically.

Specified by:

`compareTo` in interface `Comparable<Character>`

Parameters:

anotherCharacter - the Character to be compared.

Returns:

the value 0 if the argument Character is equal to this Character; a value less than 0 if this Character is numerically less than the Character argument; and a value greater than 0 if this Character is numerically greater than the Character argument (unsigned comparison). Note that this is strictly a numerical comparison; it is not locale-dependent.

Since:

1.2

compare

```
public static int compare(char x,
                          char y)
```

Compares two char values numerically. The value returned is identical to what would be returned by:

```
   Character.valueOf(x).compareTo(Character.valueOf(y))
```

Parameters:

x - the first char to compare

y - the second char to compare

Returns:

the value 0 if x == y; a value less than 0 if x < y; and a value greater than 0 if x > y

Since:

1.7

reverseBytes

```
public static char reverseBytes(char ch)
```

Returns
the value obtained by reversing the order of the bytes in the specified char value.

Parameters:

ch - The char of which to reverse the byte order.

Returns:

the value obtained by reversing (or, equivalently, swapping) the bytes in the specified char value.

Since:

1.5

getName

`public static String getName(int codePoint)`

Returns the name of the specified character `codePoint`, or null if the code point is `unassigned`.

If the specified character is not assigned a name by the *UnicodeData* file (part of the Unicode Character Database maintained by the Unicode Consortium), the returned name is the same as the result of the expression:

```
Character.UnicodeBlock.of(codePoint).toString().replace('_',
' ') + " " +
Integer.toHexString(codePoint).toUpperCase(Locale.ROOT);
```

For the `codePoints` in the *UnicodeData* file, the name returned by this method follows the naming scheme in the "Unicode Name Property" section of the Unicode Standard. For other code points, such as Hangul/Ideographs, The name generation rule above differs from the one defined in the Unicode Standard.

Parameters:

`codePoint` - the character (Unicode code point)

Returns:

the name of the specified character, or null if the code point is unassigned.

Throws:

`IllegalArgumentException` - if the specified `codePoint` is not a valid Unicode code point.

Since:

1.7

codePointOf

`public static int codePointOf(String name)`

Returns the code point value of the Unicode character specified by the given character name.

If a character is not assigned a name by the *UnicodeData* file (part of the Unicode Character Database maintained by the Unicode Consortium), its name is defined as the result of the expression:

```
Character.UnicodeBlock.of(codePoint).toString().replace('_',
' ') + " " +
Integer.toHexString(codePoint).toUpperCase(Locale.ROOT);
```

The `name` matching is case insensitive, with any leading and trailing whitespace character removed. For the code points in the *UnicodeData* file, this method recognizes the name which conforms to the name defined in the "Unicode Name Property" section in the Unicode Standard. For other code points, this method recognizes the name generated with `getName(int)` method.

Parameters:

`name` - the character name

Returns:

the code point value of the character specified by its name.

Throws:

`IllegalArgumentException` - if the specified `name` is not a valid character name.

`NullPointerException` - if `name` is `null`

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.